Supervyse OPF Intel MCTP User Guide

Revision 1.3

12/2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

Supervyse OPF Intel MCTP User Guide 6
1. Introduction 6
2. MCTP Base Protocol Service 7
2.1 Introduction 7
2.2 D-Bus Interface 8
2.2.1 Get EID list 8
2.2.2 Get MCTP endpoint information 8
2.2.3 Send an MCTP Message 10
2.2.4 Send an MCTP message and wait for the response 10
2.2.5 Trigger MCTP over SMBus Discovery 11
3. NC-SI over MCTP 11
3.1 Introduction 11
3.2 D-Bus Interface 13
3.2.1 List NC-SI package and channel of the device 13
3.2.2 Show information of the NC-SI device 13
3.2.3 List of supported NC-SI commands for NC-SI package 14
3.2.4 List of supported NC-SI commands for NC-SI channel 14
3.2.5 Issue NC-SI commands 15
List of supported NC-SI commands 15
NC-SI command response status 16
3.2.5.1 ClearInitialState (0x0) 18
3.2.5.2 SelectPackage (0x1) 19
3.2.5.3 DeselectPackage (0x2) 20
3.2.5.4 EnableChannel (0x3) 20
3.2.5.5 DisableChannel (0x4) 21
3.2.5.6 ResetChannel (0x5) 22
3.2.5.7 EnableChannelTx (0x6) 23
3.2.5.8 DisableChannelTx (0x7) 23
3.2.5.9 SetLink (0x9) 24
3.2.5.10 GetLinkStatus (0xA) 26
3.2.5.11 SetVlanFilter (0xB) 30
3.2.5.12 EnableVlan (0xC) 32
3.2.5.13 DisableVlan (0xD) 32
3.2.5.14 SetMacAddress (0xE) 33
3.2.5.15 EnableBroadcastFilter (0x10) 34
3.2.5.16 DisableBroadcastFilter (0x11) 35
3.2.5.17 GetVersionId (0x15) 36
3.2.5.18 GetCapabilities (0x16) 38
3.2.5.19 GetParameters (0x17) 42
3.2.5.20 GetCtrlPkgStatistics (0x18) 47
3.2.5.21 GetNcsiStatistics (0x19) 52
3.2.5.22 GetPassThroughStatistics (0x1A) 54
3.2.5.23 GetPackageStatus (0x1B) 56
3.2.5.24 OEMCommand (0x50) 57
4. NVMe-MI over MCTP 58
4.1 Introduction 58
4.2 Query devices supporting NVMe-MI 58
4.3 Get summary information of an NVMe-MI device 59
4.4 Issue NVMe-MI commands 62
4.4.1 Supported NVMe-MI commands 62
4.4.2 NVMe-MI response status 63
4.4.3 D-Bus methods for NVMe-MI commands 64
4.4.3.1 ControlPrimitivesPause 64
4.4.3.2 ControlPrimitivesResume 65
4.4.3.3 ControlPrimitivesAbort 65

4.4.3.4 ControlPrimitivesGetState 66
4.4.3.5 ControlPrimitivesReplay 69
4.4.3.6 MIReadDataStructureSubsystem 70
4.4.3.7 MIReadDataStructurePort 71
4.4.3.8 MIReadDataStructureControllerList 73
4.4.3.9 MIReadDataStructureControllerInfo 74
4.4.3.10 MIReadDataStructureOptionalCmd 75
4.4.3.11 MISubsystemHealthStatusPoll 78
4.4.3.12 MIControllerHealthStatusPoll 81
4.4.3.13 MIConfigurationSetI2CFreq 85
4.4.3.14 MIConfigurationSetHealthStatusChange 85
4.4.3.15 MIConfigurationSetMCTPTUSize 87
4.4.3.16 MIConfigurationGetI2CFreq 87
4.4.3.17 MIConfigurationGetMCTPTUSize 88
4.4.3.18 MIVPDRead 89
4.4.3.19 MIVPDWrite 90
4.4.3.20 ACIdentify 91
4.4.3.21 ACGetFeatures 92
4.4.3.22 ACGetLogPage 94
5. MCTP Wrapper 96
5.1 Introduction 96
5.2 MCTP Wrapper API 96
5.2.1 MCTP_BINDING_ID 96
5.2.2 MCTP_MESSAGE_TYPE 97
5.2.3 MCTPW_EVENT 97
5.2.4 MCTPW_STATUS 97
5.2.5 MCTPW_LOG 97
5.2.6 Function 98
5.2.6.1 mctpw_new 98
5.2.6.2 mctpw_set_device_update_callback 98
5.2.6.3 mctpw_set_rx_message_callback 98
5.2.6.4 mctpw_set_user_data 99
5.2.6.5 mctpw_set_log_level 99
5.2.6.6 mctpw_get_ep_list 99
5.2.6.7 mctpw_send 100
5.2.6.8 mctpw_send_receive 100
5.2.6.9 mctpw_delete 101
5.2.6.10 mctpw_device_update_cb 101
5.2.6.11 mctpw_rx_message_cb 101
5.3 Example Code 102

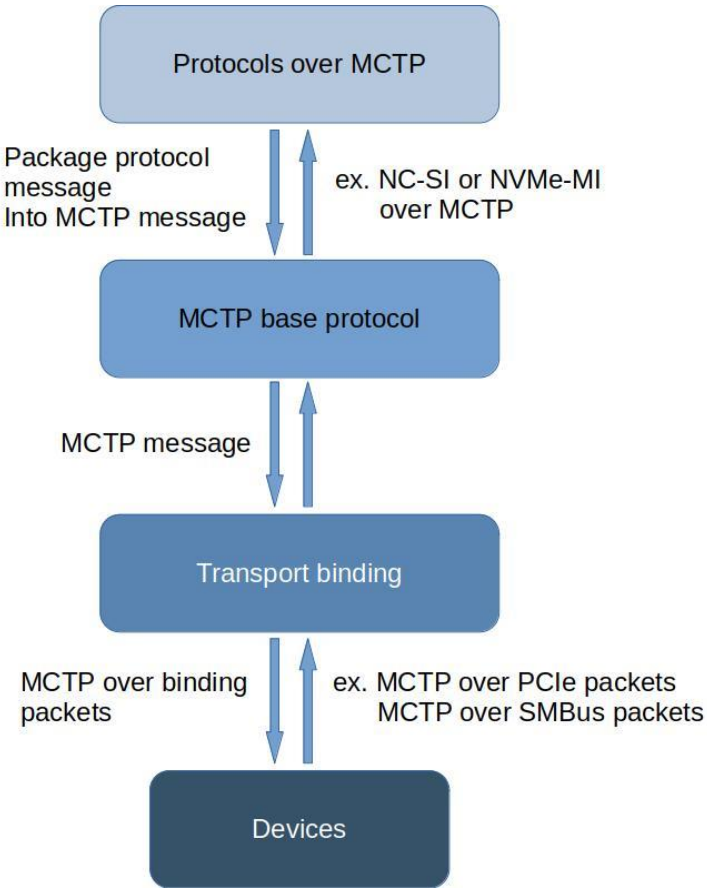
Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2024-06-07	Jim Pai	Will Tsai	RV22.11 Release Document
1.1	2024-06-07	Will Tsai	Jim Pai	Remove AMD part
1.2	2024-09-26	Jason Lin	Jim Pai	1.Remove ncsiservice 2.Add ncsidaemon
1.3	2024-12-31	Will Tsai	Jim Pai	Refine the document

Supervyse OPF Intel MCTP User Guide

1. Introduction

Management Component Transport Protocol (MCTP) is a protocol designed by the Distributed Management Task Force (DMTF). MCTP is intended to be used for intercommunication between BMC and managed devices. MCTP can potentially be used on different bus types, like PCIe or SMBus.



The MCTP message will pass through layers during transmission from the sender to the receiver:

- Protocol over MCTP

  A protocol that is designed to use MCTP to transmit protocol messages. The protocol message will be encapsulated in the payload of an MCTP message. For example, NC-SI over MCTP is a protocol that transmits NC-SI commands using MCTP.

- MCTP Base protocol

  The protocol is responsible to route MCTP-formatted messages from source to destination.

- Transport binding

Encapsulate MCTP messages into the packet format that can be transmitted on a specific physical interface. Transport binding also defines how to transmit these packets over the interface. For example, PCIe transport binding defines how to transmit MCTP over PCIe packets on PCIe bus.

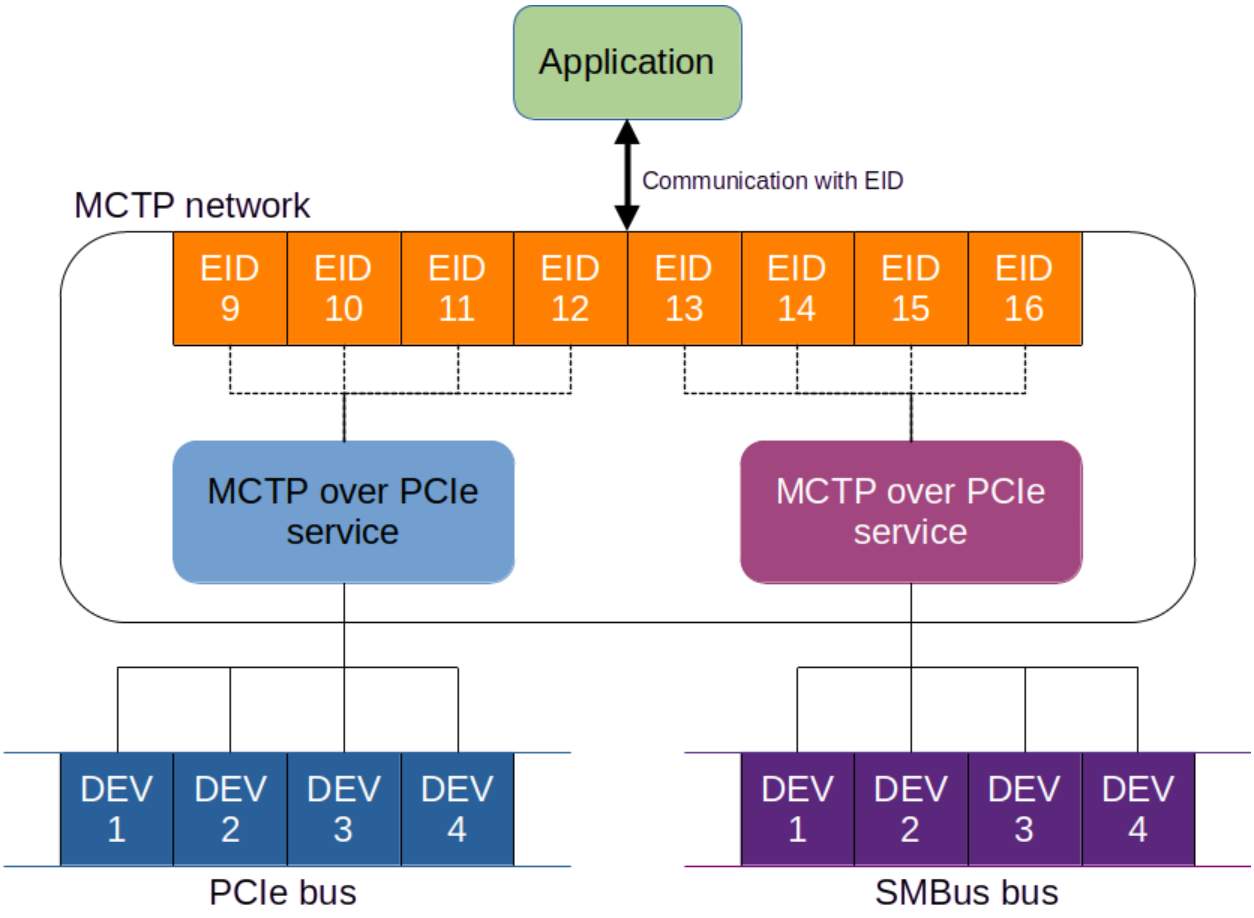
MCTP constructs a network that enables devices within the network to communicate with each other. In the MCTP network, all devices are MCTP endpoints logically addressed by MCTP endpoint IDs (EIDs), so there is no difference when sending messages to an endpoint over PCIe, SMBus, or other physical interfaces.

This user guide introduces the functionality of services that implement the MCTP base protocol, NC-SI, and NVMe-MI, as well as the MCTP wrapper, which is a shared library that facilitates MCTP application development.

2. MCTP Base Protocol Service

2.1 Introduction

- MCTP base protocol service provides functions to query discovered endpoints, get endpoint information, and communicate with endpoints



- All devices in MCTP network are addressed with EIDs, which means users need to know the discovered EIDs before transmitting messages the device over MCTP

- If users want to communicate with the devices on the PCIe bus, they need to first query the EID of devices from the MCTP over PCIe service. Similarly, the same applies to SMBus.
- MCTP base protocol service supports MCTP over PCIe and MCTP over SMBus

2.2 D-Bus Interface

- All functions of MCTP base protocol service are accessed via D-Bus
- Basic information about the MCTP D-Bus service
  - D-Bus service name
    - MCTP over PCIe: xyz.openbmc_project.MCTP_PCIe
    - MCTP over SMBus: xyz.openbmc_project.MCTP_SMBus_PCIe_slot
  - D-Bus interface: xyz.openbmc_project.MCTP.Base
    - Supported D-Bus methods
      - SendMctpMessagePayload
      - SendReceiveMctpMessagePayload
      - TriggerDeviceDiscovery

2.2.1 Get EID list

- Discovered MCTP endpoints will be exposed as D-Bus objects /xyz/openbmc_project/mctp/device/X
  - X is endpoint ID (EID)
- Users can check EID list from D-Bus object tree
- Command format

```
busctl tree $MCTP_SERVICE_NAME
```

  - MCTP_SERVICE_NAME could be
    - xyz.openbmc_project.MCTP_PCIe for MCTP over PCIe
    - xyz.openbmc_project.MCTP_SMBus_PCIe_slot for MCTP over SMBus
- Example command and result

```
# busctl tree xyz.openbmc_project.MCTP_PCIe
`-/xyz
  `-/xyz/openbmc_project
    `-/xyz/openbmc_project/mctp
      `-/xyz/openbmc_project/mctp/device
        `-/xyz/openbmc_project/mctp/device/9
```

  - In this example, an EID 9 is discovered from MCTP over PCIe

2.2.2 Get MCTP endpoint information

- Select the D-Bus object of the endpoint from section 2.2.1 to check its information
- Command format

```
busctl introspect $MCTP_SERVICE_NAME $ENDPOINT_DBUS_OBJECT
```

- Example command to get information of an MCTP over PCIe endpoint

```
# busctl introspect xyz.openbmc_project.MCTP_PCIe \
```

```
/xyz/openbmc_project/mctp/device/9
NAME                                          TYPE      SIGNATURE RESULT/VALUE
org.freedesktop.DBus.Introspectable           interface -         -
.Introspect                                   method    -         s
org.freedesktop.DBus.Peer                     interface -         -
.GetMachineId                                 method    -         s
.Ping                                         method    -         -
org.freedesktop.DBus.Properties               interface -         -
.Get                                          method    ss        v
.GetAll                                       method    s         a{sv}
.Set                                          method    ssv       -
.PropertiesChanged                            signal    sa{sv}as  -
xyz.openbmc_project.Common.UUID               interface -         -
.UUID                                         property  s         "00000000-000
xyz.openbmc_project.MCTP.Endpoint             interface -         -
.Mode                                         property  s         "xyz.openbmc_
.NetworkId                                    property  q         0
xyz.openbmc_project.MCTP.PCIVendorDefined     interface -         -
.MessageTypeProperty                          property  aq        1 46106
.VendorID                                     property  s         "0x2210"
xyz.openbmc_project.MCTP.SupportedMessageTypes interface -        -
.Ethernet                                     property  b         false
.MctpControl                                  property  b         true
.NCSI                                         property  b         false
.NVMeMgmtMsg                                  property  b         false
.PLDM                                         property  b         false
.SPDM                                         property  b         false
.VDIANA                                       property  b         false
.VDPCI                                        property  b         true
```

- Example command to get information of an MCTP over SMBus endpoint

```
# busctl introspect xyz.openbmc_project.MCTP_SMBus_PCIe_slot \
/xyz/openbmc_project/mctp/device/9
NAME                                          TYPE      SIGNATURE RESULT/VALUE
org.freedesktop.DBus.Introspectable           interface -         -
.Introspect                                   method    -         s
org.freedesktop.DBus.Peer                     interface -         -
.GetMachineId                                 method    -         s
.Ping                                         method    -         -
org.freedesktop.DBus.Properties               interface -         -
.Get                                          method    ss        v
.GetAll                                       method    s         a{sv}
.Set                                          method    ssv       -
.PropertiesChanged                            signal    sa{sv}as  -
xyz.openbmc_project.Common.UUID               interface -         -
.UUID                                         property  s         "ffffffff-fff
xyz.openbmc_project.MCTP.Endpoint             interface -         -
.Mode                                         property  s         "xyz.openbmc_
.NetworkId                                    property  q         0
xyz.openbmc_project.MCTP.SupportedMessageTypes interface -        -
.Ethernet                                     property  b         false
.MctpControl                                  property  b         true
.NCSI                                         property  b         true
.NVMeMgmtMsg                                  property  b         false
.PLDM                                         property  b         false
.SPDM                                         property  b         false
.VDIANA                                       property  b         false
.VDPCI                                        property  b         false
```

- Property definition

Property	Definition
UUID	MCTP endpoint UUID
Mode	MCTP role of this endpoint
NetworkId	MCTP network ID
Ethernet	Endpoint supports Ethernet over MCTP message type
MctpControl	Endpoint supports MCTP Control message type
NCSI	Endpoint supports NC-SI message type
NVMeMgmtMsg	Endpoint supports NVMe-MI message type
PLDM	Endpoint supports PLDM message type
SPDM	Endpoint supports SPDM message type
VDIANA	Endpoint supports Vendor Defined – PCI message type
VDPCI	Endpoint supports Vendor Defined – IANA message type

2.2.3 Send an MCTP Message

- Send an MCTP message to an endpoint
- D-Bus method format
  - D-Bus object: /xyz/openbmc_project/mctp
  - D-Bus interface: xyz.openbmc_project.MCTP.Base
  - D-Bus method: SendMctpMessagePayload
  - Parameter

Type	Definition
byte	Destination EID of the endpoint to send
byte	MCTP message tag of the message
boolean	MCTP tag owner of the message
array:byte	MCTP message payload

- Example command to send an MCTP message to MCTP over SMBus EID 9

```
# dbus-send --system --print-reply --type=method_call \
--dest=xyz.openbmc_project.MCTP_SMBus_PCIe_slot /xyz/openbmc_project/mctp \
xyz.openbmc_project.MCTP.Base.SendMctpMessagePayload \
byte:9 byte:0 boolean:true array:byte:0x00,0x80,0x05
method return time=1644043374.251887 sender=:1.109 -> destination=:1.48479
serial=221049 reply_serial=2
   int32 0
```

2.2.4 Send an MCTP message and wait for the response

- Send an MCTP message to an endpoint and wait until the response is received or timeout
- D-Bus method format
  - D-Bus object: /xyz/openbmc_project/mctp
  - D-Bus interface: xyz.openbmc_project.MCTP.Base
  - D-Bus method: SendReceiveMctpMessagePayload
  - Parameter

Type	Definition
byte	Destination EID of the endpoint to send
array:byte	MCTP message payload
uint16	Timeout waiting for the response

- Response

Type	Definition
array:byte	MCTP response message byte array

- Example command to send an MCTP message to MCTP over PCIe EID 9 and wait for the response

```
# dbus-send --system --print-reply --type=method_call \
--dest=xyz.openbmc_project.MCTP_PCIe /xyz/openbmc_project/mctp \
xyz.openbmc_project.MCTP.Base.SendReceiveMctpMessagePayload \
byte:9 array:byte:0x00,0x80,0x05 uint16:240
method return time=1644043762.026761 sender=:1.111 -> destination=:1.48658
serial=115 reply_serial=2
   array of bytes [
      00 00 05 00 01 7e
   ]
```

2.2.5 Trigger MCTP over SMBus Discovery

- Normally, MCTP over SMBus service will periodically discover devices, but users can trigger the discovery immediately
- D-Bus method format
  - D-Bus object: /xyz/openbmc_project/mctp
  - D-Bus interface: xyz.openbmc_project.MCTP.Base
  - D-Bus method: TriggerDeviceDiscovery
- Example command to trigger MCTP over SMBus discovery

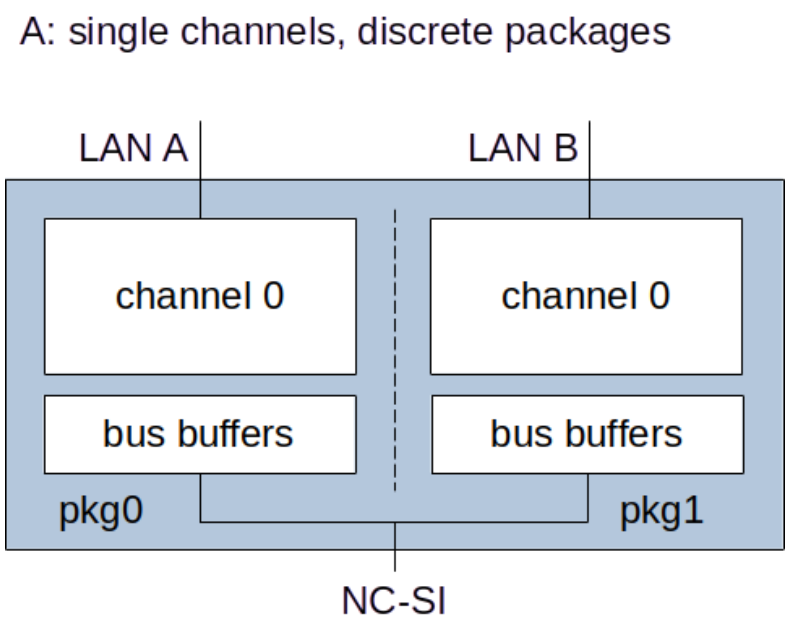
```
# dbus-send --system --print-reply --type=method_call \
--dest=xyz.openbmc_project.MCTP_SMBus_PCIe_slot /xyz/openbmc_project/mctp \
xyz.openbmc_project.MCTP.Base.TriggerDeviceDiscovery
```

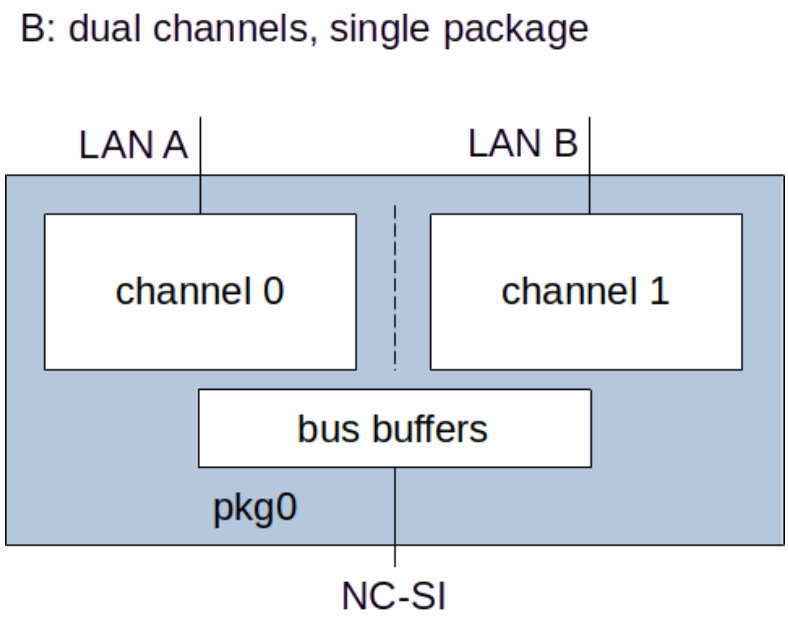
3. NC-SI over MCTP

3.1 Introduction

Network Controller Sideband Interface (NC-SI) is to define an interoperable sideband communication interface standard to enable the exchange of management data between the Management Controller (BMC) and Network Controller. NC-SI also defines the protocol and commands necessary for the operation of the sideband communication interface.

Below figure illustrates 2 examples of Network Controller configurations:





- Package

  One or more NC-SI channels in a Network Controller that share a common set of electrical buffers and common electrical buffer controls for the NC-SI bus. Typically, a single, logical NC-SI package exists for a single physical Network Controller package (chip or module). However, this specification allows a single physical chip or module to hold multiple NC-SI logical packages.

- Channel

  The control logic and data paths through a single network interface (port). A Network Controller that has multiple network interface ports can support an equivalent number of NC-SI channels.

ncsi-daemon is a service implements mandatory commands defined by NC-SI specification. ncsi-aemon also provides the functions to get endpoints supporting NC-SI message type and to issue NC-SI commands. It uses Insyde's MCTP wrapper to discover and communicate with NC-SI devices. Users could use D-Bus interface to utilize functions of ncsi-daemon.

3.2 D-Bus Interface

- ncsi-daemon D-Bus service name
  - xyz.openbmc_project.NCSIDaemon

3.2.1 List NC-SI package and channel of the device

- Command and example response

```
# busctl tree xyz.openbmc_project.NCSIDaemon
`-/xyz
  `-/xyz/openbmc_project
    `-/xyz/openbmc_project/ncsi
      `-/xyz/openbmc_project/ncsi/PCIE_10
        `-/xyz/openbmc_project/ncsi/PCIE_10/package0
          `-/xyz/openbmc_project/ncsi/PCIE_10/package0/0
```

- Definition of the D-Bus object path
  - /xyz/openbmc_project/ncsi/**PCIE_10**
    - This object contains the information of the device
    - **PCIE** mean the device is discovered via MCTP over PCIe
    - **10** is the EID of the device
  - /xyz/openbmc_project/ncsi/PCIE_10/**package0**
    - This object contains the information of an NC-SI package
    - **package0** is the package at index 0
  - /xyz/openbmc_project/ncsi/PCIE_10/package0/**0**
    - This object contains information of an NC-SI channel
    - The number **0** at the end of path is channel index

3.2.2 Show information of the NC-SI device

- Use busctl introspect to check the device D-Bus object found in 3.2.1
- Properties of this NC-SI device are listed under interface **xyz.openbmc_project.NCSIdaemon.device**
- Command and example result

```
# busctl introspect xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10
NAME                                TYPE      SIGNATURE RESULT/VALUE
org.freedesktop.DBus.Introspectable interface -         -
.Introspect                         method    -         s
org.freedesktop.DBus.Peer           interface -         -
.GetMachineId                       method    -         s
.Ping                               method    -         -
org.freedesktop.DBus.Properties     interface -         -
.Get                                method    ss        v
.GetAll                             method    s         a{sv}
.Set                                method    ssv       -
.PropertiesChanged                  signal    sa{sv}as  -
xyz.openbmc_project.NCSIdaemon.device interface -         -
```

```
.DeviceId                                  property  q         5427
.DeviceType                                property  s         "MCTP over PCIE"
.Eid                                       property  u         10
.MacAddress                                property  ay        6 160 54 159 112 15 12
.Present                                   property  b         true
.VendorId                                  property  q         32902
```

3.2.3 List of supported NC-SI commands for NC-SI package

- Use busctl introspect to check the package found in 3.2.1
- The NC-SI commands this package supports are listed under interface **xyz.openbmc_project.NCSIdaemon.command**
- Command and example result

```
# busctl introspect xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0
NAME                                       TYPE      SIGNATURE RESULT/VALUE FLAGS
org.freedesktop.DBus.Introspectable        interface -         -            -
.Introspect                                method    -         s            -
org.freedesktop.DBus.Peer                  interface -         -            -
.GetMachineId                              method    -         s            -
.Ping                                      method    -         -            -
org.freedesktop.DBus.Properties            interface -         -            -
.Get                                       method    ss        v            -
.GetAll                                    method    s         a{sv}        -
.Set                                       method    ssv       -            -
.PropertiesChanged                         signal    sa{sv}as  -            -
xyz.openbmc_project.NCSIdaemon.command     interface -         -            -
.DeselectPackage                           method    -         a{sv}        -
.GetPackageStatus                          method    -         a{sv}        -
.OEMCommand                                method    ay        a{sv}        -
.SelectPackage                             method    b         a{sv}        -
```

3.2.4 List of supported NC-SI commands for NC-SI channel

- Use busctl introspect to check the channel found in 3.2.1
- The NC-SI commands this channel supports are listed under interface **xyz.openbmc_project.NCSIdaemon.command**
- Command and example result

```
# busctl introspect xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0
NAME                                       TYPE      SIGNATURE RESULT/VALUE FLAGS
org.freedesktop.DBus.Introspectable        interface -         -            -
.Introspect                                method    -         s            -
org.freedesktop.DBus.Peer                  interface -         -            -
.GetMachineId                              method    -         s            -
.Ping                                      method    -         -            -
org.freedesktop.DBus.Properties            interface -         -            -
.Get                                       method    ss        v            -
.GetAll                                    method    s         a{sv}        -
.Set                                       method    ssv       -            -
.PropertiesChanged                         signal    sa{sv}as  -            -
xyz.openbmc_project.NCSIdaemon.channel     interface -         -            -
.Channel                                   property  i         0            emits-ch
.Package                                   property  i         0            emits-ch
xyz.openbmc_project.NCSIdaemon.command     interface -         -            -
```

```
.ClearInitialState                    method    -          a{sv}        -
.DisableBroadcastFilter               method    -          a{sv}        -
.DisableChannel                       method    b          a{sv}        -
.DisableChannelTx                     method    -          a{sv}        -
.DisableVlan                          method    -          a{sv}        -
.EnableBroadcastFilter                method    bbbb       a{sv}        -
.EnableChannel                        method    -          a{sv}        -
.EnableChannelTx                      method    -          a{sv}        -
.EnableVlan                           method    y          a{sv}        -
.GetCapabilities                      method    -          a{sv}        -
.GetCtrlPkgStatistics                 method    -          a{sv}        -
.GetLinkStatus                        method    -          a{sv}        -
.GetNcsiStatistics                    method    -          a{sv}        -
.GetParameters                        method    -          a{sv}        -
.GetPassThroughStatistics             method    -          a{sv}        -
.GetVersionId                         method    -          a{sv}        -
.OEMCommand                           method    ay         a{sv}        -
.ResetChannel                         method    -          a{sv}        -
.SetLink                              method    byybbbyu   a{sv}        -
.SetMacAddress                        method    byyay      a{sv}        -
.SetVlanFilter                        method    qbyby      a{sv}        -
```

3.2.5 Issue NC-SI commands

List of supported NC-SI commands

- Based on Network Controller Sideband Interface (NC-SI) Specification Version: 1.1.1

ID	M/O[1]	Package Command[2]	Command	Description
0	M	x	ClearInitialState	Used by the Management Controller to acknowledge that the Network Controller is in the Initial State
1	O	v	SelectPackage	Used to explicitly select a controller package to transmit packets through the NC-SI interface
2	O	v	DeselectPackage	Used to explicitly instruct the controller package to stop transmitting packets through the NC-SI interface
3	M	x	EnableChannel	Used to enable the NC-SI channel and to cause the forwarding of bidirectional Management Controller packets to start
4	M	x	DisableChannel	Used to disable the NC-SI channel and to cause the forwarding of bidirectional Management Controller packets to cease
5	M	x	ResetChannel	Used to synchronously put the Network Controller back to the Initial State
6	M	x	EnableChannelTx	Used to explicitly enable the channel to transmit Pass-

				through packets onto the network
7	M	x	DisableChannelTx	Used to explicitly disable the channel from transmitting Pass-through packets onto the network
9	M	x	SetLink	Used during OS absence to force link settings, or to return to auto-negotiation mode
A	M	x	GetLinkStatus	Used to get current link status information
B	M	x	SetVlanFilter	Used to program VLAN IDs for VLAN filtering
C	M	x	EnableVlan	Used to enable VLAN filtering of Management Controller RX packets
D	M	x	DisableVlan	Used to disable VLAN filtering
E	M	x	SetMacAddress	Used to configure and enable unicast and multicast MAC address filters
10	M	x	EnableBroadcastFilter	Used to enable selective broadcast packet filtering
11	M	x	DisableBroadcastFilter	Used to disable all broadcast packet filtering, and to enable the forwarding of all broadcast packets
15	M	x	GetVersionId	Used to get controller-related version information
16	M	x	GetCapabilities	Used to get optional functions supported by the NC-SI
17	M	x	GetParameters	Used to get configuration parameter values currently in effect on the controller
18	O	x	GetCtrlPkgStatistics	Used to get current packet statistics for the Ethernet Controller
19	O	x	GetNcsiStatistics	Used to request the packet statistics specific to the NC-SI
1A	O	x	GetPassThroughStatistics	Used to request NC-SI Pass-through packet statistics
1B	O	v	GetPackageStatus	Used to get current status of the package
50	O	v	OEMCommand	Used to request vendor-specific data
1. M = Mandatory (required), O = Optional 2. The commands are addressed to a package				

NC-SI command response status

- Besides NC-SI response payload, response of each NC-SI DBus calls will have a Status field to indicate error states

Status Value	Description
1	Success
0	General fail
-1	Invalid parameter
-2	Payload length error
-3	Wrong Message type
-4	Eid is invalid
-5	Checksum error
-6	Command id is not match
-7	IID is not match
-8	Interface Initialization Required
-100	Device not response / not exist

- NC-SI standard response code is in Response code field

Value	Description	Comment
0x0000	Command Completed	Returned for succeessful command completion. When this response code is returned, the reason code shall be 0x0000
0x0001	Command Failed	Returned to report that a valid command could not be processed or failed to complete correctly
0x0002	Command Unavailable	Returned to report that a command is temporarily unavailable for execution because the controller is in a transient state or busy condition
0x0003	Command Unsupported	Returned to report that a command is not supported by the implementation. The reason code "Unknown / Unsupported Command Type should be returned along with this response code for all unsupported commands.
0x8000-0xFFFF	Vendor/OEM-specific	Response codes defined by the vendor of the controller

- NC-SI standard reason code is in Reason code field

Value	Description	Comment
0x0000	No Error/No Reason Code	When used with the Command Completed response code, indicates that the command completed normally. Otherwise this value indicates that no

		additional reason code information is being provided.
0x0001	Interface Initialization Required	Returned for all commands except Select/Deselect Package commands when the channel is in the Initial State, until the channel receives a Clear Initial State command
0x0002	Parameter Is Invalid, Unsupported, or Out-of-Range	Returned when a received parameter value is outside of the acceptable values for that parameter
0x0003	Channel Not Ready	May be returned when the channel is in a transient state in which it is unable to process commands normally
0x0004	Package Not Ready	May be returned when the package and channels within the package are in a transient state in which normal command processing cannot be done
0x0005	Invalid payload length	The payload length in the command is incorrect for the given command
0x7FFF	Unknown / Unsupported Command Type	Returned when the command type is unknown or unsupported. This reason code shall only be used when the response code is 0x0003 (Command Unsupported)
0x8000-0xFFFF	OEM Reason Code	Vendor-specific reason code defined by the vendor of the controller

3.2.5.1 ClearInitialState (0x0)

- Description : Used by the Management Controller to acknowledge that the Network Controller is in the Initial State
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
```

```
xyz.openbmc_project.NCSIdaemon.command ClearInitialState -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.2 SelectPackage (0x1)

- Description : Used to explicitly select a controller package to transmit packets through the NC-SI interface (The NC-SI package in the Network Controller shall also become selected if the package receives any other NC-SI command that is directed to the package or to a channel within the package)
- Object : package
- Request parameter :

Field	Type	Description
Hardware Arbitration Disable	b	0b = Hardware arbitration between packages is enabled. 1b = Disable hardware arbitration.

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0 \
xyz.openbmc_project.NCSIdaemon.command SelectPackage b 0 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
```

```
                                        "type" : "q",
                                        "data" : 0
                                },
                                "Response code" : {
                                        "type" : "q",
                                        "data" : 0
                                }
                        }
                ]
}
```

3.2.5.3 DeselectPackage (0x2)

- Description : Used to explicitly instruct the controller package to stop transmitting packets through the NC-SI interface
- Object : package
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0 \
xyz.openbmc_project.NCSIdaemon.command DeselectPackage -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.4 EnableChannel (0x3)

- Description : enable the Network Controller to allow transmission of Pass-through and AEN packets to the Management Controller through the NC-SI.
- Object : channel

- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command EnableChannel -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.5 DisableChannel (0x4)

- Description : allows the Management Controller to disable the flow of packets, including Pass-through and AEN, to the Management Controller
- Object : channel
- Request parameter :

Field	Type	Description
Allow Link Down	b	0b = Keep link up (establish and/or keep a link established) while channel is disabled 1b = Allow link to be taken down while channel is disabled

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command DisableChannel b 1 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.6 ResetChannel (0x5)

- Description : Used to synchronously put the Network Controller back to the Initial State
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command ResetChannel -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
```

```
                        }
                }
        ]
}
```

3.2.5.7 EnableChannelTx (0x6)

- Description : Used to explicitly enable the channel to transmit Pass-through packets onto the network
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command EnableChannelTx -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.8 DisableChannelTx (0x7)

- Description : Used to explicitly disable the channel from transmitting Pass-through packets onto the network
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table

Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command DisableChannelTx -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.9 SetLink (0x9)

- Description : Used during OS absence to force link settings, or to return to auto-negotiation mode
- Object : channel
- Request parameter :

Field	Type	Description
Auto Negotiation	b	1b = enable 0b = disable
Link Speed Selection[1]	y	Bit 00: 1b = enable 10 Mbps Bit 01: 1b = enable 100 Mbps Bit 02: 1b = enable 1000 Mbps (1 Gbps) Bit 03: 1b = enable 10 Gbps Bit 04: 1b = enable 20 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 05: 1b = enable 25 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 06: 1b = enable 40 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0)
Duplex Setting	y	Bit 00: 1b = enable half-duplex Bit 01: 1b = enable full-duplex
Pause Capability	b	1b = disable 0b = enable
Asymmetric Pause Capability	b	1b = enable 0b = disable

OEM Link Settings Field Valid (see Table 42)	b	1b = enable 0b = disable
Additional Link Speeds[1] (see Link Speed Selection)	y	Bit 0: 1b = enable 50 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 1: 1b = enable 100 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 2: 1b = enable 2.5 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 3: 1b = enable 5 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0)
OEM Link Settings	u	Vendor specified
1. This parameter is shifted to bit 0 for convenience in input, which differs from the specification definition		

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Specific reason code:

Value	Description	Comment
0x0901	Set Link Host OS/ Driver Conflict	Returned when the Set Link command is received when the Host NC driver is operational
0x0902	Set Link Media Conflict	Returned when Set Link command parameters conflict with the media type (for example, Fiber Media)
0x0903	Set Link Parameter Conflict	Returned when Set Link parameters conflict with each other (for example, 1000 Mbps HD with copper media)
0x0904	Set Link Power Mode Conflict	Returned when Set Link parameters conflict with current low-power levels by exceeding capability
0x0905	Set Link Speed Conflict	Returned when Set Link parameters attempt to force more than one speed at the same time
0x0906	Link Command Failed-Hardware Access Error	Returned when PHY R/W access fails to complete normally while executing the Set Link or Get Link Status command

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command SetLink \
byybbbyu 1 0x7f 2 1 0 0 1 0 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 0
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 2305
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 1
                        }
                }
        ]
}
```

3.2.5.10 GetLinkStatus (0xA)

- Description : Used to get current link status information
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Link Flag	b	0b = Link is down 1b = Link is up (including Low Power Idle state in EEE)
Speed and duplex	y	0x0 = Auto-negotiate not complete [per IEEE 802.3], or SerDes Flag = 1b, or no Highest Common Denominator (HCD) from the following options (0x1 through 0xF) was found. 0x1 = 10BASE-T half-duplex 0x2 = 10BASE-T full-duplex 0x3 = 100BASE-TX half-duplex 0x4 = 100BASE-T4 0x5 = 100BASE-TX full-duplex 0x6 = 1000BASE-T half-duplex 0x7 = 1000BASE-T full-duplex

		0x8 = 10G-BASE-T support or 10 Gbps 0x9 = 20 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xA = 25 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xB = 40 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xC = 50 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xD = 100 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xE = 2.5 Gbps (optional for NC-SI 1.1, RESERVED for NCSI 1.0) 0xF = Use values defined in Enhanced Speed and Duplex field starting at bit 24 (optional for NC-SI 1.1, RESERVED for NC-SI 1.0)
Auto Negotiate Flag	b	1b = Auto-negotiation is enabled.
Auto Negotiate Complete	b	1b = Auto-negotiation has completed.
Parallel Detection Flag	b	1b = Link partner did not support auto-negotiation and parallel detection was used to get link.
Link Partner Advertised Speed and Duplex 1000TFD	b	1b = Link Partner is 1000BASE-T full-duplex capable.
Link Partner Advertised Speed and Duplex 1000THD	b	1b = Link Partner is 1000BASE-T half-duplex capable.
Link Partner Advertised Speed 100T4	b	1b = Link Partner is 100BASE-T4 capable.
Link Partner Advertised Speed and Duplex 100TXFD	b	1b = Link Partner is 100BASE-TX full-duplex capable.
Link Partner Advertised Speed and Duplex 100TXHD	b	1b = Link Partner is 100BASE-TX half-duplex capable.
Link Partner Advertised Speed and Duplex 10TFD	b	1b = Link Partner is 10BASE-T full-duplex capable.
Link Partner Advertised Speed and Duplex 10THD	b	1b = Link Partner is 10BASE-T half-duplex capable.
TX Flow Control Flag	b	0b = Transmission of Pause frames by the NC onto the external network interface is disabled. 1b = Transmission of Pause frames by the NC onto the external network interface is enabled.
RX Flow Control Flag	b	0b = Reception of Pause frames by the NC from the external network interface is disabled. 1b = Reception of Pause frames by the NC from the external network interface is enabled.
Link Partner Advertised Flow Control	y	00b = Link partner is not pause capable. 01b = Link partner supports symmetric pause.

		10b = Link partner supports asymmetric pause toward link partner. 11b = Link partner supports both symmetric and asymmetric pause.
SerDes Link	b	0b = SerDes not used or used to connect to an external PHY 1b = SerDes used as a direct attach interface
OEM Link Speed Valid	b	0b = OEM link settings are invalid. 1b = OEM link settings are valid.
Extended Speed and duplex	y	0x0 = Auto-negotiate not complete [per IEEE 802.3], or SerDes Flag = 1b, or no highest common denominator speed from the following options (0x01 through 0x0F) was found. 0x01 = 10BASE-T half-duplex 0x02 = 10BASE-T full-duplex 0x03 = 100BASE-TX half-duplex 0x04 = 100BASE-T4 0x05 = 100BASE-TX full-duplex 0x06 = 1000BASE-T half-duplex 0x07 = 1000BASE-T full-duplex 0x08 = 10G-BASE-T support or 10 Gbps 0x09 = 20 Gbps 0x0A = 25 Gbps 0x0B = 40 Gbps 0x0C = 50 Gbps 0x0D = 100 Gbps 0x0E = 2.5 Gbps 0x0F = 5 Gbps 0x10-0xFF = Reserved
Host NC Driver Status Indication	b	0b = The Network Controller driver for the host external network interface associated with this channel is not operational (not running), unknown, or not supported. 1b = The Network Controller driver for the host external network interface associated with this channel is being reported as operational (running).
OEM Link Status	u	OEM specific

- Specific reason code:

Value	**Description**	**Comment**

0x0A06	Link Command Failed-Hardware Access Error	Returned when PHY R/W access fails to complete normally while executing the Set Link or Get Link Status command

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetLinkStatus -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Link Flag" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Speed and duplex" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "Auto Negotiate Flag" : {
                                "type" : "b",
                                "data" : true
                        },
                        "Auto Negotiate Complete" : {
                                "type" : "b",
                                "data" : true
                        },
                        "Parallel Detection Flag" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 1000TFD" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 1000THD" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed 100T4" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 100TXFD" : {
```

```
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 100TXHD" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 10TFD" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Partner Advertised Speed and Duplex 10THD" : {
                                "type" : "b",
                                "data" : true
                        },
                        "TX Flow Control Flag" : {
                                "type" : "b",
                                "data" : false
                        },
                        "RX Flow Control Flag" : {
                                "type" : "b",
                                "data" : true
                        },
                        "Link Partner Advertised Flow Control" : {
                                "type" : "y",
                                "data" : 3
                        },
                        "SerDes Link" : {
                                "type" : "b",
                                "data" : false
                        },
                        "OEM Link Speed Valid" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Extended Speed and duplex" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "Host NC Driver Status Indication" : {
                                "type" : "b",
                                "data" : true
                        },
                        "OEM Link Status" : {
                                "type" : "u",
                                "data" : 0
                        }
                }
        ]
}
```


3.2.5.11 SetVlanFilter (0xB)

- Description : Used to program VLAN IDs for VLAN filtering (Refer to dsp0222 Figure 9 – NC-SI packet filtering flowchart)
- Object : channel
- Request parameter :

Field	Type	Description

VLAN ID	q	12 bits valid
CFI	b	Canonical Format Indicator
User Priority	y	3 bits valid
Enable	b	0b Disable this VLAN filter 1b Enable this VLAN filter
Filter Selector	y	1 Settings for VLAN filter number 1 2 Settings for VLAN filter number 2 .. N Settings for VLAN filter number N

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Specific reason code:

Value	Description	Comment
0x0B07	VLAN Tag Is Invalid	Returned when the VLAN ID is invalid (VLAN ID = 0)

- Example:

Setting VLAN filter number 1 with VLAN id 100, CFI disable, User priority 0x000 and disable

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command SetVlanFilter \
qbyby 100 0 0 0 1 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.12 EnableVlan (0xC)

- Description : Used to enable VLAN filtering of Management Controller RX packets (Refer to dsp0222 Figure 9 – NC-SI packet filtering flowchart)
- Object : channel
- Request parameter :

Field	Type	Description
Mode	y	0x00 Reserved 0x01 VLAN only 0x02 VLAN + non-VLAN 0x03 Any VLAN + non-VLAN 0x04 – 0xFF Reserved

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command EnableVlan y 3 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

It could use “GetCapabilities” command to verify

3.2.5.13 DisableVlan (0xD)

- Description : Used to disable VLAN filtering
- Object : channel
- Request parameter : None

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command DisableVlan -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.14 SetMacAddress (0xE)

- Description : Used to configure and enable unicast and multicast MAC address filters (Refer to dsp0222 Figure 9 – NC-SI packet filtering flowchart)
- Object : channel
- Request parameter :

Field	Type	Description
Enable	b	0b Disable this MAC address filter 1b Enable this MAC address filter
Address Type	y	0x0 Unicast MAC address 0x1 Multicast MAC address 0x2–0x7 Reserved
MAC Address Num	y	0x01 Configure MAC address filter number 1 … N Configure MAC address filter number N
MAC Address	ay	Mac5, mac4, mac3, mac2, mac1, mac0

- Response :

Field	Type	Description
Status	i	Refer to Status definition table

Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Specific reason code:

Value	Description	Comment
0x0E08	MAC Address Is Zero	Returned when the Set MAC Address command is received with the MAC address set to 0

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command SetMacAddress \
byyay 1 1 1 6 0x5 0x4 0x3 0x2 0x1 0x0 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.15 EnableBroadcastFilter (0x10)

- Description : Used to enable selective broadcast packet filtering (Refer to dsp0222 Figure 9 – NC-SI packet filtering flowchart)
- Object : channel
- Request parameter :

Field	Type	Description
ARP Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Client Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Server Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
NetBIOS Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command EnableBroadcastFilter bbbb 1 0 1 0 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.16 DisableBroadcastFilter (0x11)

- Description : Used to disable all broadcast packet filtering, and to enable the forwarding of all broadcast packets
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command DisableBroadcastFilter -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
```

```
                                        "type" : "i",
                                        "data" : 1
                                },
                                "Reason code" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "Response code" : {
                                        "type" : "q",
                                        "data" : 0
                                }
                        }
                ]
}
```

3.2.5.17 GetVersionId (0x15)

- Description : Used to get controller-related version information
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Major	y	NC-SI Version field
Minor	y	NC-SI Version field
Update	y	NC-SI Version field
Alpha1	y	NC-SI Version field
Alpha2	y	NC-SI Version field
Firmware Name String	s	Firmware Name String
Firmware Version	s	Firmware Version
PCI DID	q	PCI DID
PCI VID	q	PCI VID
PCI SSID	q	PCI SSID
PCI SVID	q	PCI SVID
Manufacturer ID (IANA)	u	The Manufacturer ID

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetVersionId -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
```

```
                                        "data" : 1
                                },
                                "Reason code" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "Response code" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "Major" : {
                                        "type" : "y",
                                        "data" : 241
                                },
                                "Minor" : {
                                        "type" : "y",
                                        "data" : 240
                                },
                                "Update" : {
                                        "type" : "y",
                                        "data" : 240
                                },
                                "Alpha1" : {
                                        "type" : "y",
                                        "data" : 0
                                },
                                "Alpha2" : {
                                        "type" : "y",
                                        "data" : 0
                                },
                                "Firmware Name String" : {
                                        "type" : "s",
                                        "data" : "I210  FW ver"
                                },
                                "Firmware Version" : {
                                        "type" : "s",
                                        "data" : "01.07.00.03"
                                },
                                "PCI DID" : {
                                        "type" : "q",
                                        "data" : 5427
                                },
                                "PCI VID" : {
                                        "type" : "q",
                                        "data" : 32902
                                },
                                "PCI SSID" : {
                                        "type" : "q",
                                        "data" : 1
                                },
                                "PCI SVID" : {
                                        "type" : "q",
                                        "data" : 32902
                                },
                                "Manufacturer ID (IANA)" : {
                                        "type" : "u",
                                        "data" : 343
                                }
                        }
                ]
}
```

3.2.5.18 GetCapabilities (0x16)

- Description : Used to get optional functions supported by the NC-SI
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Hardware Arbitration Capability	b	0b = Hardware arbitration capability is not supported by the package. 1b = Hardware arbitration capability is supported by the package.
Host NC Driver Status	b	0b = Host NC Driver Indication status is not supported. 1b = Host NC Driver Indication status is supported.
Network Controller to Management Controller Flow Control Support	b	0b = Network Controller to Management Controller flow control is not supported. 1b = Network Controller to Management Controller flow control is supported.
Management Controller to Network Controller Flow Control Support	b	0b = Management Controller to Network Controller flow control is not supported. 1b = Management Controller to Network Controller flow control is supported.
All multicast addresses support	b	0b = The channel cannot accept all multicast addresses. The channel does not support enable/disable global multicast commands. 1b = The channel can accept all multicast addresses. The channel supports enable/disable global multicast commands.
Hardware Arbitration Implementation Status	y	00b = Unknown 01b = Hardware arbitration capability is not implemented for the package on the given system. 10b = Hardware arbitration capability is implemented for the package on the given system.

		11b = Reserved.
ARP Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Client Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Server Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
NetBIOS Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
IPv6 Neighbor Advertisement	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
IPv6 Router Advertisement	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCPv6 relay and server multicast		1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCPv6 multicasts from server to clients listening on wellknown UDP ports	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
IPv6 MLD	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
IPv6 Neighbor Solicitation	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
Buffering Capability	u	The amount of buffering in bytes that the channel provides for inbound packets destined for the Management Controller
Link Status Change AEN control	b	0b = Disable Link Status Change AEN 1b = Enable Link Status Change AEN
Configuration Required AEN control	b	0b = Disable Configuration Required AEN 1b = Enable Configuration Required AEN
Host NC Driver Status Change AEN control	b	0b = Disable Host NC Driver Status Change AEN 1b = Enable Host NC Driver Status Change AEN
OEM-specific AEN control	q	OEM-specific control
VLAN Filter Count	y	The number of VLAN filters, up to 15, that the channel supports, as defined by the Set VLAN Filter command.
Mixed Filter Count	y	The number of mixed address filters that the channel supports.
Multicast Filter Count	y	The number of multicast MAC address filters that the channel supports.
Unicast Filter Count	y	The number of unicast MAC address filters that the channel supports.
VLAN only	b	1 = VLAN shall be supported in the implementation.
VLAN + non-VLAN	b	0 = Filtering 'VLAN + non-VLAN' traffic is not supported in the implementation. 1 = Filtering 'VLAN + non-VLAN' traffic is supported in the implementation.

Any VLAN + non-VLAN	b	0 = Filtering 'Any VLAN + non-VLAN' traffic is not supported in the implementation. 1 = Filtering 'Any VLAN + non-VLAN' traffic is supported in the implementation.
Channel Count	y	The number of channels supported by the Network Controller.

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetCapabilities -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Hardware Arbitration Capability" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Host NC Driver Status" : {
                                "type" : "b",
                                "data" : true
                        },
                        "Network Controller to Management Controller Flow
                       Control Support" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Management Controller to Network Controller Flow
                       Control Support" : {
                                "type" : "b",
                                "data" : false
                        },
                        "All multicast addresses support" : {
                                "type" : "b",
                                "data" : true
                        },
                        "Hardware Arbitration Implementation Status" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "ARP Packets" : {
                                "type" : "b",
                                "data" : true
                        },
```

```
"DHCP Client Packets" : {
        "type" : "b",
        "data" : true
},
"DHCP Server Packets" : {
        "type" : "b",
        "data" : true
},
"NetBIOS Packets" : {
        "type" : "b",
        "data" : true
},
"IPv6 Neighbor Advertisement" : {
        "type" : "b",
        "data" : true
},
"IPv6 Router Advertisement" : {
        "type" : "b",
        "data" : true
},
"DHCPv6 relay and server multicast" : {
        "type" : "b",
        "data" : true
},
"DHCPv6 multicasts from server to clients listening
on wellknown UDP ports" : {
        "type" : "b",
        "data" : false
},
"IPv6 MLD" : {
        "type" : "b",
        "data" : false
},
"IPv6 Neighbor Solicitation" : {
        "type" : "b",
        "data" : false
},
"Buffering Capability" : {
        "type" : "u",
        "data" : 8176
},
"Link Status Change AEN control" : {
        "type" : "b",
        "data" : true
},
"Configuration Required AEN control" : {
        "type" : "b",
        "data" : true
},
"Host NC Driver Status Change AEN control" : {
        "type" : "b",
        "data" : true
},
"OEM-specific AEN control" : {
        "type" : "q",
        "data" : 0
},
"VLAN Filter Count" : {
        "type" : "y",
        "data" : 8
},
"Mixed Filter Count" : {
```

```
                                        "type" : "y",
                                        "data" : 2
                                },
                                "Multicast Filter Count" : {
                                        "type" : "y",
                                        "data" : 0
                                },
                                "Unicast Filter Count" : {
                                        "type" : "y",
                                        "data" : 0
                                },
                                "VLAN only" : {
                                        "type" : "b",
                                        "data" : true
                                },
                                "VLAN + non-VLAN" : {
                                        "type" : "b",
                                        "data" : false
                                },
                                "Any VLAN + non-VLAN" : {
                                        "type" : "b",
                                        "data" : true
                                },
                                "Channel Count" : {
                                        "type" : "y",
                                        "data" : 1
                                }
                        }
                ]
}
```


3.2.5.19 GetParameters (0x17)

- Description : Used to get configuration parameter values currently in effect on the controller
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
MAC Address Count	y	The number of MAC addresses supported by the channel
MAC Address Flags	y	The enable/disable state for each supported MAC address
VLAN Tag Count	y	The number of VLAN Tags supported by the channel
VLAN Tag Flags	q	The enable/disable state for each supported VLAN Tag
Auto Negotiation	b	1b = enable 0b = disable
Link Speed Selection	y	Bit 00: 1b = enable 10 Mbps Bit 01: 1b = enable 100 Mbps

		Bit 02: 1b = enable 1000 Mbps (1 Gbps) Bit 03: 1b = enable 10 Gbps Bit 04: 1b = enable 20 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 05: 1b = enable 25 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 06: 1b = enable 40 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0)
Duplex Setting	y	Bit 00: 1b = enable half-duplex Bit 01: 1b = enable full-duplex
Pause Capability	b	1b = disable 0b = enable
Asymmetric Pause Capability	b	1b = enable 0b = disable
OEM Link Settings Field Valid	b	1b = enable 0b = disable
Additional Link Speeds	y	Bit 00: 1b = enable 50 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 01: 1b = enable 100 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 02: 1b = enable 2.5 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0) Bit 03: 1b = enable 5 Gbps (optional for NC-SI 1.1, RESERVED for NC-SI 1.0)
ARP Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Client Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
DHCP Server Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
NetBIOS Packets	b	1b = Forward this packet type to the Management Controller. 0b = Filter out this packet type.
Broadcast Packet Filter status	b	0b = Disabled 1b = Enabled
Channel Enabled	b	0b = Disabled 1b = Enabled
Channel Network TX Enabled	b	0b = Disabled 1b = Enabled
Global Multicast Packet Filter Status	b	0b = Disabled 1b = Enabled
VLAN Mode	y	0x00 Reserved 0x01 VLAN only 0x02 VLAN + non-VLAN 0x03 Any VLAN + non-VLAN 0x04 – 0xff Reserved

Flow Control Enable	y	0x0 Disables NC-SI flow control 0x1 Enables Network Controller to Management Controller flow control frames (Network Controller generates flow control frames) 0x2 Enables Management Controller to Network Controller flow control frames (Network Controller accepts flow control frames) 0x3 Enables bi-directional flow control frames 0x4..0xFF Reserved
Link Status Change AEN control	b	0b = Disable Link Status Change AEN 1b = Enable Link Status Change AEN
Configuration Required AEN control	b	0b = Disable Configuration Required AEN 1b = Enable Configuration Required AEN
Host NC Driver Status Change AEN control	b	0b = Disable Host NC Driver Status Change AEN 1b = Enable Host NC Driver Status Change AEN
OEM-specific AEN control	q	OEM-specific control
MAC Address 1	ay	The current contents of up to eight 6-byte MAC address filter values.
...		
MAC Address n (n <= 8)		
VLAN Tag 1	q	The current contents of up to 15 16-bit VLAN Tag filter values
...		
VLAN Tag n (n <= 15)		

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetParameters -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "MAC Address Count" : {
                                "type" : "y",
                                "data" : 2
                        },
                        "MAC Address Flags" : {
                                "type" : "y",
                                "data" : 1
                        },
```

```
                        "VLAN Tag Count" : {
                                "type" : "y",
                                "data" : 8
                        },
                        "VLAN Tag Flags" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Auto Negotiation" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Link Speed Selection" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "Duplex Setting" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "Pause Capability" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Asymmetric Pause Capability" : {
                                "type" : "b",
                                "data" : false
                        },
                        "OEM Link Settings Field Valid" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Additional Link Speeds" : {
                                "type" : "y",
                                "data" : 0
                        },
                        "ARP Packets" : {
                                "type" : "b",
                                "data" : false
                        },
                        "DHCP Client Packets" : {
                                "type" : "b",
                                "data" : false
                        },
                        "DHCP Server Packets" : {
                                "type" : "b",
                                "data" : false
                        },
                        "NetBIOS Packets" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Broadcast Packet Filter status" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Channel Enabled" : {
                                "type" : "b",
                                "data" : false
                        },
                        "Channel Network TX Enabled" : {
                                "type" : "b",
```

```
                "data" : false
        },
        "Global Multicast Packet Filter Status" : {
                "type" : "b",
                "data" : true
        },
        "VLAN Mode" : {
                "type" : "y",
                "data" : 0
        },
        "Flow Control Enable" : {
                "type" : "y",
                "data" : 0
        },
        "Link Status Change AEN control" : {
                "type" : "b",
                "data" : false
        },
        "Configuration Required AEN control" : {
                "type" : "b",
                "data" : false
        },
        "Host NC Driver Status Change AEN control" : {
                "type" : "b",
                "data" : false
        },
        "OEM-specific AEN control" : {
                "type" : "q",
                "data" : 0
        },
        "MAC Address 0" : {
                "type" : "ay",
                "data" : [
                        5,
                        4,
                        3,
                        2,
                        1,
                        0
                ]
        },
        "MAC Address 1" : {
                "type" : "ay",
                "data" : [
                        0,
                        0,
                        0,
                        0,
                        0,
                        0
                ]
        },
        "VLAN Tag 0" : {
                "type" : "q",
                "data" : 0
        },
        "VLAN Tag 1" : {
                "type" : "q",
                "data" : 0
        },
        "VLAN Tag 2" : {
                "type" : "q",
```

```
                                        "data" : 0
                                },
                                "VLAN Tag 3" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "VLAN Tag 4" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "VLAN Tag 5" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "VLAN Tag 6" : {
                                        "type" : "q",
                                        "data" : 0
                                },
                                "VLAN Tag 7" : {
                                        "type" : "q",
                                        "data" : 0
                                }
                        }
                ]
}
```

3.2.5.20 GetCtrlPkgStatistics (0x18)

- Description : Used to get current packet statistics for the Ethernet Controller
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Counters Cleared From Last Read	t	Indicate in the Counters Cleared from Last Read fields whether the corresponding field has been cleared by means other than NC-SI (possibly by the host) since it was last read by means of the NC-SI
Total Bytes Received	t	Counts the number of bytes received
Total Bytes Transmitted	t	Counts the number of bytes transmitted
Total Unicast Packets Received	t	Counts the number of good (FCS valid) packets received that passed L2 filtering by a specific MAC address
Total Multicast Packets Received	t	Counts the number of good (FCS valid) multicast packets received
Total Broadcast Packets Received	t	Counts the number of good (FCS valid) broadcast packets received
Total Unicast Packets Transmitted	t	Counts the number of good (FCS valid) packets transmitted that passed L2 filtering by a specific MAC address

Total Multicast Packets Transmitted	t	Counts the number of good (FCS valid) multicast packets transmitted
Total Broadcast Packets Transmitted	t	Counts the number of good (FCS valid) broadcast packets transmitted
FCS Receive Errors	u	Counts the number of receive packets with FCS errors
Alignment Errors	u	Counts the number of receive packets with alignment errors
False Carrier Detections	u	Counts the false carrier errors reported by the PHY
Runt Packets Received	u	Counts the number of received frames that passed address filtering, were less than minimum size (64 bytes from through , inclusively), and had a valid FCS
Jabber Packets Received	u	Counts the number of received frames that passed address filtering, were greater than the maximum size, and had a valid FCS
Pause XON Frames Received	u	Counts the number of XON packets received from the network
Pause XOFF Frames Received	u	Counts the number of XOFF packets received from the network
Pause XON Frames Transmitted	u	Counts the number of XON packets transmitted to the network
Pause XOFF Frames Transmitted	u	Counts the number of XOFF packets transmitted to the network
Single Collision Transmit Frames	u	Counts the number of times that a successfully transmitted packet encountered a single collision
Multiple Collision Transmit Frames	u	Counts the number of times that a transmitted packet encountered more than one collision but fewer than 16
Late Collision Frames	u	Counts the number of collisions that occurred after one slot time (defined by IEEE 802.3)
Excessive Collision Frames	u	Counts the number of times that 16 or more collisions occurred on a single transmit packet
Control Frames Received	u	Counts the number of MAC control frames received that are not XON or XOFF flow control frames
64-Byte Frames Received	u	Counts the number of good packets received that are exactly 64 bytes (from through , inclusively) in length
65-127 Byte Frames Received	u	Counts the number of good packets received that are 65–127 bytes (from through , inclusively) in length
128-255 Byte Frames Received	u	Counts the number of good packets received that are 128–255 bytes (from through , inclusively) in length
256-511 Byte Frames Received	u	Counts the number of good packets received that are 256–511 bytes (from through , inclusively) in length
512-1023 Byte Frames Received	u	Counts the number of good packets received that are 512–1023 bytes (from through , inclusively) in length
1024-1522 Byte Frames Received	u	Counts the number of good packets received that are 1024–1522 bytes (from through , inclusively) in length
1523-9022 Byte Frames Received	u	Counts the number of received frames that passed address filtering and were greater than 1523 bytes in length

64-Byte Frames Transmitted	u	Counts the number of good packets transmitted that are exactly 64 bytes (from through , inclusively) in length
65-127 Byte Frames Transmitted	u	Counts the number of good packets transmitted that are 65–127 bytes (from through , inclusively) in length
128-255 Byte Frames Transmitted	u	Counts the number of good packets transmitted that are 128–255 bytes (from through , inclusively) in length
256-511 Byte Frames Transmitted	u	Counts the number of good packets transmitted that are 256–511 bytes (from through , inclusively) in length
512-1023 Byte Frames Transmitted	u	Counts the number of good packets transmitted that are 512–1023 bytes (from through , inclusively) in length
1024-1522 Byte Frames Transmitted	u	Counts the number of good packets transmitted that are 1024–1522 bytes (from through , inclusively) in length
1523-9022 Byte Frames Transmitted	u	Counts the number of transmitted frames that passed address filtering and were greater than 1523 in length
Valid Bytes Received	t	Counts the bytes received in all packets that did not manifest any type of error
Error Runt Packets Received	u	Counts the number of invalid frames that were less than the minimum size (64 bytes from through , inclusively)
Error Jabber Packets Received	u	Counts Jabber packets, which are defined as packets that exceed the programmed MTU size and have a bad FCS value

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetCtrlPkgStatistics -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Counters Cleared From Last Read" : {
                                "type" : "t",
                                "data" : 517116
                        },
                        "Total Bytes Received" : {
                                "type" : "t",
                                "data" : 18446744073709551615
```

```
                        },
                        "Total Bytes Transmitted" : {
                                "type" : "t",
                                "data" : 18446744073709551615
                        },
                        "Total Unicast Packets Received" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Total Multicast Packets Received" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Total Broadcast Packets Received" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Total Unicast Packets Transmitted" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Total Multicast Packets Transmitted" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Total Broadcast Packets Transmitted" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "FCS Receive Errors" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Alignment Errors" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "False Carrier Detections" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Runt Packets Received" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Jabber Packets Received" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pause XON Frames Received" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Pause XOFF Frames Received" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Pause XON Frames Transmitted" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Pause XOFF Frames Transmitted" : {
```

```
                "type" : "u",
                "data" : 0
        },
        "Single Collision Transmit Frames" : {
                "type" : "u",
                "data" : 0
        },
        "Multiple Collision Transmit Frames" : {
                "type" : "u",
                "data" : 0
        },
        "Late Collision Frames" : {
                "type" : "u",
                "data" : 4294967295
        },
        "Excessive Collision Frames" : {
                "type" : "u",
                "data" : 4294967295
        },
        "Control Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "64-Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "65-127 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "128-255 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "256-511 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "512-1023 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "1024-1522 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "1523-9022 Byte Frames Received" : {
                "type" : "u",
                "data" : 4294967295
        },
        "64-Byte Frames Transmitted" : {
                "type" : "u",
                "data" : 4294967295
        },
        "65-127 Byte Frames Transmitted" : {
                "type" : "u",
                "data" : 4294967295
        },
        "128-255 Byte Frames Transmitted" : {
                "type" : "u",
                "data" : 4294967295
```

```
                        },
                        "256-511 Byte Frames Transmitted" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "512-1023 Byte Frames Transmitted" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "1024-1522 Byte Frames Transmitted" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "1523-9022 Byte Frames Transmitted" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Valid Bytes Received" : {
                                "type" : "t",
                                "data" : 18446744073709551615
                        },
                        "Error Runt Packets Received" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Error Jabber Packets Received" : {
                                "type" : "u",
                                "data" : 4294967295
                        }
                }
        ]
}
```


3.2.5.21 GetNcsiStatistics (0x19)

- Description : Used to request the packet statistics specific to the NC-SI
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
NC-SI Commands Received	u	For packets that are not dropped, this field returns the number of NC-SI Control packets received and identified as NC-SI commands.
NC-SI Control Packets Dropped	u	Counts the number of NC-SI Control packets that were received and dropped (Packets with correct FCS and EtherType, but are dropped for one of the other reasons listed in 6.9.2.1). NC-SI Control Packets that were dropped because the channel ID was not valid may not be included in this statistics counter.

NC-SI Command Type Errors (NC-SI Unsupported Commands Received)	u	Counts the number of NC-SI command packets that were received, but are not supported. (Network controller responded to the command with a Command Unsupported response code).
NC-SI Command Checksum Errors	u	Counts the number of NC-SI Control Packets that were received but dropped because of an invalid checksum (if checksum is provided and checksum validation is supported by the channel)
NC-SI Receive Packets	u	Counts the total number of NC-SI Control packets received. This count is the sum of NC-SI Commands Received and NC-SI Control Packets Dropped.
NC-SI Transmit Packets	u	Counts the total number of NC-SI Control packets transmitted to the Management Controller. This count is the sum of NC-SI responses sent and AENs sent.
AENs Sent	u	Counts the total number of AEN packets transmitted to the Management Controller

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetNcsiStatistics -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "NC-SI Commands Received" : {
                                "type" : "u",
                                "data" : 29
                        },
                        "NC-SI Control Packets Dropped" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "NC-SI Command Type Errors" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "NC-SI Command Checksum Errors" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "NC-SI Receive Packets" : {
                                "type" : "u",
                                "data" : 29
```

```
                        },
                        "NC-SI Transmit Packets" : {
                                "type" : "u",
                                "data" : 29
                        },
                        "AENs Sent" : {
                                "type" : "u",
                                "data" : 0
                        }
                }
        ]
}
```

3.2.5.22 GetPassThroughStatistics (0x1A)

- Description : Used to request NC-SI Pass-through packet statistics
- Object : channel
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Pass-through TX Packets Received on NC-SI Interface	t	Counts the number of Pass-through packets forwarded by the channel to the LAN
Pass-through TX Packets Dropped	u	Counts the number of Pass-through packets from the Management Controller that were dropped by the Network Controller
Pass-through TX Packet Channel State Errors	u	Counts the number of egress management packets (Management Controller to Network Controller) that were dropped because the channel was in the disabled state when the packet was received
Pass-through TX Packet Undersized Errors	u	Counts the number of Pass-through packets from the Management Controller that were undersized (under 64 bytes, including FCS)
Pass-through TX Packet Oversized Errors	u	Counts the number of Pass-through packets from the Management Controller that were oversized (over 1522 bytes, including FCS)
Pass-through RX Packets Received on LAN Interface	u	Counts the number of Pass-through packets that were received on the LAN interface of the channel. This counter does not necessarily count the number of packets that were transmitted to the Management Controller, because some of the packets might have been dropped due to RX queue overflow.
Total Pass-through RX Packets Dropped	u	Counts the number of Pass-through packets that were received on the LAN interface of the channel but were dropped and not transmitted to the Management Controller
Pass-through RX Packet Channel State Errors	u	Counts the number of ingress management packets (channel to Management Controller) that were dropped because the channel was in the disabled state when the packet was received. The

		NC may also count packets that were dropped because the package was in the deselected state.
Pass-through RX Packet Undersized Errors	u	Counts the number of Pass-through packets from the LAN that were undersized (under 64 bytes, including FCS)
Pass-through RX Packet Oversized Errors	u	Counts the number of Pass-through packets from the LAN that were oversized (over 1522 bytes, including FCS)

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0/0 \
xyz.openbmc_project.NCSIdaemon.command GetPassThroughStatistics -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Pass-through TX Packets Received on NC-SI
                       Interface" : {
                                "type" : "t",
                                "data" : 0
                        },
                        "Pass-through TX Packets Dropped" : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Pass-through TX Packet Channel State Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pass-through TX Packet Undersized Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pass-through TX Packet Oversized Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pass-through RX Packets Received on LAN Interface"
                       : {
                                "type" : "u",
                                "data" : 0
                        },
                        "Total Pass-through RX Packets Dropped" : {
                                "type" : "u",
                                "data" : 0
                        },
```

```
                        "Pass-through RX Packet Channel State Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pass-through RX Packet Undersized Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        },
                        "Pass-through RX Packet Oversized Errors" : {
                                "type" : "u",
                                "data" : 4294967295
                        }
                }
        ]
}
```

3.2.5.23 GetPackageStatus (0x1B)

- Description : Used to get current status of the package
- Object : package
- Request parameter : None
- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Hardware Arbitration Status	b	0b = Hardware arbitration is non-operational (inactive) or unsupported. 1b = Hardware arbitration is supported, active, and implemented for the package on the given system.

- Example:

```
# busctl call xyz.openbmc_project.NCSIDaemon \
/xyz/openbmc_project/ncsi/PCIE_10/package0 \
xyz.openbmc_project.NCSIdaemon.command GetPackageStatus -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Hardware Arbitration Status" : {
                                "type" : "b",
                                "data" : false
```

```
                        }
                }
        ]
}
```

3.2.5.24 OEMCommand (0x50)

- Description : Used to request vendor-specific data
- Object : package/channel (Depends on command)
- Request parameter :

Field	Type	Description
Oem command request	ay	Oem command request payload

- Response :

Field	Type	Description
Status	i	Refer to Status definition table
Reason code	q	Refer to NC-SI standard reason code
Response code	q	Refer to NC-SI standard response code
Oem command response	ay	payload (exclude reason/response code)

- Example:

Mellanox Get PF MAC Address

```
# busctl call xyz.openbmc_project.NCSIDaemon
/xyz/openbmc_project/ncsi/Smbus_100/package0/0
xyz.openbmc_project.NCSIdaemon.command OEMCommand ay 8 0x00 0x00 0x81 0x19
0x00 0x00 0x00 0x00 -j
{
        "type" : "a{sv}",
        "data" : [
                {
                        "Status" : {
                                "type" : "i",
                                "data" : 1
                        },
                        "Reason code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Response code" : {
                                "type" : "q",
                                "data" : 0
                        },
                        "Oem command response payload" : {
                                "type" : "ay",
                                "data" : [
                                        0,
                                        0,
                                        129,
                                        25,
                                        0,
```

```
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0,
                                                0
                                        ]
                                }
                        }
                ]
}
```

4. NVMe-MI over MCTP

4.1 Introduction

NVMe Management Interface (NVMe-MI) is used for BMC to communicate out-of-band with NVMe devices over MCTP. nvmemid is the service which implements the mandatory commands defined by the NVMe-MI specification. Users can use D-Bus interface to utilize the functions of nvmemid, including querying NVMe-MI devices, sending NVMe-MI commands and receiving responses.

Normally, nvmemid is used to provide the functionality for Redfish to collect information about storage devices supporting NVMe-MI, but users can utilize nvmemid directly to ensure the data is correct before passing to Redfish.

nvmemid is implemented based on NVM Express Management Interface Revision 1.0 and only supports NVMe-MI over MCTP over SMBus.

4.2 Query devices supporting NVMe-MI

- Users can query discovered NVMe-MI devices from MCTP over SMBus with nvmemid
- Get device list with D-Bus method "GetDeviceList"
  - D-Bus service name: xyz.openbmc_project.NVMeMIDaemon
  - D-Bus object path: /xyz/openbmc_project/nvmemi
  - D-Bus interface: xyz.openbmc_project.NVMeMIDaemon.NVMeMI
  - D-Bus method: GetDeviceList
- "GetDeviceList" will return an array of bytes that each byte presents the MCTP EID of an NVMe-MI device
  - Example command and result

```
# busctl call xyz.openbmc_project.NVMeMIDaemon /xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI GetDeviceList -j
{
        "type" : "ay",
        "data" : [
                [
                        105
                ]
        ]
```

```
}
```

- In above example, an NVMe-MI device is found, and its EID is 105

4.3 Get summary information of an NVMe-MI device

- "Summary Information" collects properties related to a storage device by issuing several NVMe-MI commands to the targeted device
  - Collected properties

Property	Description
AssetTag	A unique identifier for the device
CapacityBytes	Capacity of the device in bytes
Manufacturer	Manufacturer of the device
MediaType	Media type of the device
Model	Model of device
NegotiatedSpeedGbs	The port's PCIe negotiated link speed
Oem.CompositeTemperature	current temperature in degrees Celsius
Oem.Functional	The device is functional
Oem.PCIe0LinkSpeed	The port's PCIe negotiated link speed
Oem.PCIe0LinkWidth	The negotiated PCIe link width for this port
Oem.PCIe1LinkSpeed	Not used
Oem.PCIe1LinkWidth	Not used
Oem.PCIeBusNumber	PCI bus number
Oem.PCIeDeviceID	PCI device ID
Oem.PCIeLinkActive	If the first port's PCIe link is up
Oem.PCIeSubsystemDeviceID	PCI Subsystem Device ID
Oem.PCIeSubsystemVendorID	PCI Subsystem Vendor ID
Oem.PCIeVendorID	PCI Vendor ID
Oem.Powered	The device is functional
Oem.ResetRequired	The device needs a reset to resume normal operation
Oem.SmartWarning.AvailableSpareThreshold	When the Available Spare falls below the threshold indicated in this field
Oem.SmartWarning.MediaReadOnly	The device has been placed in read only mode
Oem.SmartWarning.ReliabilityDegraded	The device reliability has been degraded due to significant media related errors or any internal error
Oem.SmartWarning.TemperatureThreshold	The device temperature is: a) greater than or equal to an over temperature threshold; or b) less than or equal to an under temperature threshold
Oem.SmartWarning.VolatileMemoryBackupFailed	The volatile memory backup device has failed

PartNumber	Product Part/Model Number
PredictedMediaLifeLeftPercent	The percentage of device life left
Protocol	The operational protocol of device
Revision	Firmware revision
SerialNumber	Serial number of the device
Status.Health	Device health status
Status.HealthRollup	Device health rollup status
Status.State	The device is functional
StatusIndicator"	Overall device status

- Get summary information of a device with D-Bus method "GetSummaryInfo"
  - ☐ D-Bus service name: xyz.openbmc_project.NVMeMIDaemon
  - ☐ D-Bus object path: /xyz/openbmc_project/nvmemi
  - ☐ D-Bus interface: xyz.openbmc_project.NVMeMIDaemon.NVMeMI
  - ☐ D-Bus method: GetSummaryInfo
  - ☐ Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device to get the summary information. For available EIDs, refer to section 4.2

  - ☐ Example command and result

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.GetSummaryInfo byte:105
method return time=1734881754.635954 sender=:1.51 -> destination=:1.8229
serial=27 reply_serial=2
   array [
      dict entry(
         string "AssetTag"
         variant             string ""
      )
      dict entry(
         string "CapacityBytes"
         variant             uint64 3200631791616
      )
      dict entry(
         string "Manufacturer"
         variant             string "WestDigi"
      )
      dict entry(
         string "MediaType"
         variant             string "SSD"
      )
      dict entry(
         string "Model"
         variant             string "ESSDRIVE"
      )
      dict entry(
         string "NegotiatedSpeedGbs"
         variant             double 8
      )
      dict entry(
         string "Oem.CompositeTemperature"
```

```
            variant             double 30
        )
        dict entry(
            string "Oem.Functional"
            variant             int32 1
        )
        dict entry(
            string "Oem.PCIe0LinkSpeed"
            variant             double 8
        )
        dict entry(
            string "Oem.PCIe0LinkWidth"
            variant             string "PCIe x4"
        )
        dict entry(
            string "Oem.PCIe1LinkSpeed"
            variant             double -1
        )
        dict entry(
            string "Oem.PCIe1LinkWidth"
            variant             string "N/A"
        )
        dict entry(
            string "Oem.PCIeBusNumber"
            variant             string "0x00"
        )
        dict entry(
            string "Oem.PCIeDeviceID"
            variant             string "0x2500"
        )
        dict entry(
            string "Oem.PCIeLinkActive"
            variant             int32 1
        )
        dict entry(
            string "Oem.PCIeSubsystemDeviceID"
            variant             string "0x0000"
        )
        dict entry(
            string "Oem.PCIeSubsystemVendorID"
            variant             string "0x1B96"
        )
        dict entry(
            string "Oem.PCIeVendorID"
            variant             string "0x1B96"
        )
        dict entry(
            string "Oem.Powered"
            variant             int32 1
        )
        dict entry(
            string "Oem.ResetRequired"
            variant             int32 0
        )
        dict entry(
            string "Oem.SmartWarning.AvailableSpareThreshold"
            variant             int32 0
        )
        dict entry(
            string "Oem.SmartWarning.MediaReadOnly"
            variant             int32 0
        )
```

```
        dict entry(
           string "Oem.SmartWarning.ReliabilityDegraded"
           variant             int32 0
        )
        dict entry(
           string "Oem.SmartWarning.TemperatureThreshold"
           variant             int32 0
        )
        dict entry(
           string "Oem.SmartWarning.VolatileMemoryBackupFailed"
           variant             int32 0
        )
        dict entry(
           string "PartNumber"
           variant             string "WUS4C6432DSP3X1"
        )
        dict entry(
           string "PredictedMediaLifeLeftPercent"
           variant             int32 100
        )
        dict entry(
           string "Protocol"
           variant             string "NVMe"
        )
        dict entry(
           string "Revision"
           variant             string "R2210010"
        )
        dict entry(
           string "SerialNumber"
           variant             string "A06A2601"
        )
        dict entry(
           string "Status.Health"
           variant             string "OK"
        )
        dict entry(
           string "Status.HealthRollup"
           variant             string "OK"
        )
        dict entry(
           string "Status.State"
           variant             string "Enabled"
        )
        dict entry(
           string "StatusIndicator"
           variant             string "OK"
        )
    ]
```

4.4 Issue NVMe-MI commands

4.4.1 Supported NVMe-MI commands

NMIMT[1]	Opcode	O/M[2]	Command
Control Primitives			
0h	00h	M	Pause

0h	01h	M	Resume
0h	02h	M	Abort
0h	03h	M	Get State
0h	04h	M	Replay
Management Interface Command Set			
1h	00h	M	Read NVMe-MI Data Structure
1h	01h	M	NVM Subsystem Health Status Poll
1h	02h	M	Controller Health Status Poll
1h	03h	M	Configuration Set
1h	04h	M	Configuration Get
1h	05h	M	VPD Read
1h	06h	M	VPD Write
NVM Express Admin Command Set			
2h	02h	M	Get Log Page
2h	06h	M	Identify
2h	0Ah	M	Get Features
NOTES: NVMe-MI Message Type Optional or Mandatory; O – optional and M - mandatory			

4.4.2 NVMe-MI response status

- Besides NVMe-MI response payload, response of each NVMe-MI D-Bus calls will have a Status field to indicate error states

Status Value	Description
00h	The command completed successfully.
01h	The command is in progress and requires more time to complete processing. When this Response Message Status value is used in a Response Message, a subsequent message contains the result of the Command Message. This Response Message Status Value shall not be sent more than once per Request Message.
02h	The command could not be executed due to a vendor specific internal error.
03h	The associated command opcode field is not valid. Invalid opcodes include reserved and optional opcodes that are not implemented.
04h	Invalid command parameter field value. Request Messages received with reserved values in defined fields shall be completed with an Invalid Parameter Error Response Message. Request Messages received with reserved or unimplemented values in defined fields shall be completed with an Invalid Parameter Error Response Message. Other error conditions that result in Invalid Parameter Error Response Message are noted elsewhere in this specification.
05h	The Command Message body was larger or smaller than that expected by the command due to a reason other than too much or too little input data (e.g., the command did not contain all the required parameters or no input data was expected but the command message body is larger than that needed to contain the required parameters). The expected command

	message body size is determined by the command opcode assuming no other errors are detected (e.g., Invalid Command Opcode or Invalid Parameter).
06h	The Command Message requires input data and contains too much or too little input data.
07h	A command was prohibited from being executed due to a vendor specific protection mechanism.
20h	More updates to the VPD are attempted than allowed.
21h	PCIe Inaccessible: The PCIe functionality is not available at this time.

4.4.3 D-Bus methods for NVMe-MI commands

4.4.3.1 ControlPrimitivesPause

- The Pause Control Primitive is used to suspend response transmission and suspend the timeout waiting for packet for both Command Slots in a Management Endpoint. The CSI field in a Pause Control Primitive is not used and shall be cleared to '0'
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
Tag	byte	Tag (TAG): This field contains an opaque value that is passed by the Management Endpoint from the Control Primitive to the associated Response Message. The Response Message contains the same value in this field as the corresponding Request Message.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Pause Flag Status Slot 0	byte	Pause Flag Status Slot 0 (PFSS0): This field indicates whether or not Command Slot 0 is paused after completing the Pause primitive. A '1' in this field indicates the Command Slot is paused. A '0' in this field indicates the Command Slot is not paused.
Pause Flag Status Slot 1	byte	Pause Flag Status Slot 1 (PFSS1): This field indicates whether or not Command Slot 1 is paused after completing the Pause primitive. A '1' in this field indicates the Command Slot is paused. A '0' in this field indicates the Command Slot is not paused.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ControlPrimitivesPause \
byte:105 byte:1
method return time=1734885746.157775 sender=:1.51 -> destination=:1.40734
serial=47 reply_serial=2
   array [
      dict entry(
         string "Pause Flag Status Slot 0"
         variant             byte 0
      )
      dict entry(
         string "Pause Flag Status Slot 1"
         variant             byte 0
```

```
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.2 ControlPrimitivesResume

- The Resume Control Primitive is used to resume from a paused state. This is the complement to the Pause Control Primitive
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device
Tag	byte	Tag (TAG): This field contains an opaque value that is passed by the Management Endpoint from the Control Primitive to the associated Response Message. The Response Message contains the same value in this field as the corresponding Request Message.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ControlPrimitivesResume \
byte:105 byte:2
method return time=1734885878.344692 sender=:1.51 -> destination=:1.41911
serial=49 reply_serial=2
   array [
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.3 ControlPrimitivesAbort

- The Resume Control Primitive is used to resume from a paused state. This is the complement to the Pause Control Primitive
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1

Tag	byte	Tag (TAG): This field contains an opaque value that is passed by the Management Endpoint from the Control Primitive to the associated Response Message. The Response Message contains the same value in this field as the corresponding Request Message.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Command Processing Abort Status	byte	This field indicates the effect of the Abort primitive on the processing of the Command Message associated with the Command Slot. 0h – Command aborted after processing completed or no command to abort. 1h – Command aborted before processing began 2h – Command processing partially completed. 3h – Reserved

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ControlPrimitivesAbort \
byte:105 byte:1 byte:3
method return time=1734885978.754282 sender=:1.51 -> destination=:1.42890
serial=51 reply_serial=2
   array [
      dict entry(
         string "Command Processing Abort Status"
         variant             byte 0
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.4 ControlPrimitivesGetState

- The Get State Control Primitive is used to check the state of a Command Slot
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
Tag	byte	Tag (TAG): This field contains an opaque value that is passed by the Management Endpoint from the Control Primitive to the associated Response Message. The Response Message contains the same value in this field as the corresponding Request Message.
CESF	byte	(CESF): This field specifies whether or not to clear the error state flags when completing this command

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Bad Header Version	byte	Bad Header Version (BHVS): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'
Bad Message Integrity Check Error	byte	Bad Message Integrity Check Error (BMICE): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
Bad Packet or Other Physical Layer	byte	Bad Packet or Other Physical Layer (BPOPL): This field is set to '1' if a packet sent to the Management Endpoint failed a transport specific packet integrity check since the last time Get State primitive was executed with the CESF field set to '1'.
Bad, Unexpected, or Expired Message Tag	byte	Bad, Unexpected, or Expired Message Tag (BUEMT): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
Command Message to non-Idle Command Slot	byte	Command Message to non-Idle Command Slot (CMNICS): This field is set to '1' if the Management Endpoint discarded one or more Command Messages due to overlapping Command Messages to a Command Slot since the last time Get State primitive was executed with the CESF field set to '1'.
Incorrect Transmission Unit	byte	Incorrect Transmission Unit (ITU): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
NVM Subsystem Reset Occurred	byte	NVM Subsystem Reset Occurred (NSSRO): This field indicates when an NVM Subsystem Reset occurs while main power is applied. This field is set to '1' if the last occurrence of an NVM Subsystem Reset occurred while main power was applied to the NVM Subsystem. This field is cleared to '0' following a power cycle and following a Get State primitive with the CESF field set to '1'.
Out-of-Sequence Packet Sequence Number	byte	Out-of-Sequence Packet Sequence Number (OSPSN): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
Pause Flag	byte	Pause Flag (PFLG): This field indicates whether or not the Command Slot is paused. A '1' in this field indicates the Command Slot is paused. A '0' in this field indicates the Command Slot is not paused. While the Pause Flag is set, the Management Endpoint disables the timeout waiting for packet timer, as defined in the MCTP Base Specification, for the Command Slot and does not transmit responses to Command Messages.

Slot Command Servicing State	byte	Slot Command Servicing State (SSTA): This field indicates the current command servicing state of the Command Slot. An implementation may choose to indicate only the Idle and Process states in this field. Refer to Figure 21. 0h: Idle 1h: Receive 2h: Process 3h: Transmit
Timeout Waiting for a Packet	byte	Timeout Waiting for a Packet (WPTT): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
Unexpected Middle or End of Packet	byte	Unsupported Transmission Unit (UTUNT): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'
Unknown Destination ID	byte	Unknown Destination ID (UDSTID): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.
Unsupported Transmission Unit	byte	Unsupported Transmission Unit (UTUNT): This field is set to '1' if the Management Endpoint detected an error of this type (refer to the MCTP Base Specification) since the last time Get State primitive was executed with the CESF field set to '1'.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ControlPrimitivesGetState \
byte:105 byte:1 byte:5 byte:0
method return time=1734886080.820228 sender=:1.51 -> destination=:1.43883
serial=53 reply_serial=2
   array [
      dict entry(
         string "Bad Header Version"
         variant             byte 0
      )
      dict entry(
         string "Bad Message Integrity Check Error"
         variant             byte 0
      )
      dict entry(
         string "Bad Packet or Other Physical Layer"
         variant             byte 0
      )
      dict entry(
         string "Bad, Unexpected, or Expired Message Tag"
         variant             byte 0
      )
      dict entry(
         string "Command Message to non-Idle Command Slot"
         variant             byte 0
      )
```

```
      dict entry(
         string "Incorrect Transmission Unit"
         variant             byte 0
      )
      dict entry(
         string "NVM Subsystem Reset Occurred"
         variant             byte 0
      )
      dict entry(
         string "Out-of-Sequence Packet Sequence Number"
         variant             byte 0
      )
      dict entry(
         string "Pause Flag"
         variant             byte 0
      )
      dict entry(
         string "Slot Command Servicing State"
         variant             byte 0
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
      dict entry(
         string "Timeout Waiting for a Packet"
         variant             byte 0
      )
      dict entry(
         string "Unexpected Middle or End of Packet"
         variant             byte 1
      )
      dict entry(
         string "Unknown Destination ID"
         variant             byte 0
      )
      dict entry(
         string "Unsupported Transmission Unit"
         variant             byte 0
      )
   ]
```

4.4.3.5 ControlPrimitivesReplay

- The Replay Control Primitive is used to retransmit the Response Message for the last Command Message processed in a Command Slot
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
Tag	byte	Tag (TAG): This field contains an opaque value that is passed by the Management Endpoint from the Control Primitive to the associated Response Message. The Response Message contains the same value in this field as the corresponding Request Message.

RRO	byte	Response Replay Offset (RRO): This field specifies the starting packet number from which the Response Message associated with the last Command Message processed in the Command Slot should be replayed.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Response Replay	byte	Response Replay (RR): This bit indicates if a previous Response Message is retransmitted. This field is set to '1' if the requested Response Message is retransmitted by the Management Endpoint. This field is cleared to '0' if the requested Response Message is not retransmitted.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ControlPrimitivesReplay \
byte:105 byte:0 byte:6 byte:0
method return time=1734886747.626596 sender=:1.51 -> destination=:1.49958
serial=55 reply_serial=2
   array [
      dict entry(
         string "Response Replay"
         variant             byte 0
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.6 MIReadDataStructureSubsystem

- The Read NVMe-MI Data Structure command requests data that describes information about the NVM Subsystem, the Management Endpoint or the NVMe Controllers
- This command sends Read NVMe-MI Data Structure request with Data Structure Type NVM Subsystem Information
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

NVMe-MI Major Version Number	byte	NVMe-MI Major Version Number (MJR): This field shall be set to 1h to indicate the major version number of this specification.
NVMe-MI Minor Version Number	byte	NVMe-MI Minor Version Number (MNR): This field shall be cleared to 0h to indicate the minor version number of this specification.
Number of Ports	byte	Number of Ports (NUMP): This field specifies the maximum number of ports of any type supported by the NVM Subsystem. This is a 0's based value.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIReadDataStructureSubsystem \
byte:105 byte:0
method return time=1734887283.176036 sender=:1.51 -> destination=:1.54879
serial=57 reply_serial=2
   array [
      dict entry(
         string "NVMe-MI Major Version Number"
         variant             byte 1
      )
      dict entry(
         string "NVMe-MI Minor Version Number"
         variant             byte 1
      )
      dict entry(
         string "Number of Ports"
         variant             byte 1
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.7 MIReadDataStructurePort

- The Read NVMe-MI Data Structure command requests data that describes information about the NVM Subsystem, the Management Endpoint or the NVMe Controllers
- This command sends Read NVMe-MI Data Structure request with Data Structure Type Port Information
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
PORTID	byte	Port Identifier (PORTID): This field contains the identifier of the port whose data structure is returned.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Maximum MCTP Transmission Unit Size	uint16	Maximum MCTP Transmission Unit Size: The maximum MCTP Transmission Unit size the port is capable of sending and receiving.
PCIe Current Link Speed	string	The port's PCIe negotiated link speed using the same encoding as the PCIe Supported Link Speed Vector field. A value of 0h in this field indicates the PCIe Link is not available.
PCIe Maximum Link Width	string	The maximum PCIe link width for this NVM Subsystem port.
PCIe Maximum Payload Size	byte	: This field indicates the Max Payload Size for the specified PCIe port. If the link is not active, this field should be cleared to 0h.
PCIe Negotiated Link Width	string	The negotiated PCIe link width for this port.
PCIe Supported Link Speeds Vector	byte	This field indicates the Supported Link Speeds for the specified PCIe port.
Port Type	string	Specifies the port type.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIReadDataStructurePort \
byte:105 byte:0 byte:0
method return time=1734887653.589902 sender=:1.51 -> destination=:1.58242
serial=59 reply_serial=2
   array [
      dict entry(
         string "Maximum MCTP Transmission Unit Size"
         variant             uint16 124
      )
      dict entry(
         string "PCIe Current Link Speed"
         variant             string "8.0 GT/s"
      )
      dict entry(
         string "PCIe Maximum Link Width"
         variant             string "PCIe x4"
      )
      dict entry(
         string "PCIe Maximum Payload Size"
         variant             byte 128
      )
      dict entry(
         string "PCIe Negotiated Link Width"
         variant             string "PCIe x4"
      )
      dict entry(
         string "PCIe Supported Link Speeds Vector"
         variant             byte 7
      )
      dict entry(
         string "Port Type"
         variant             string "PCIe"
      )
      dict entry(
         string "Status"
```

```
         variant             byte 0
      )
   ]
```

4.4.3.8 MIReadDataStructureControllerList

- The Read NVMe-MI Data Structure command requests data that describes information about the NVM Subsystem, the Management Endpoint or the NVMe Controllers
- This command sends Read NVMe-MI Data Structure request with Data Structure Type Controller List
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTRLID	byte	Controller Identifier (CTRLID): This field contains the Controller identifier whose data structure is returned.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Number of Identifiers	uint16	This field contains the number of controller entries in the list. There may be up to 2,047 identifiers in the list. A value of 0h indicates there are no controllers in the list.
Identifier 0	string	This field contains the NVM subsystem unique controller identifier for the first controller in the list, if present
...	string	...
Identifier N	string	This field contains the NVM subsystem unique controller identifier for the N+1 controller in the list, if present.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIReadDataStructureControllerList \
byte:105 byte:1 byte:0
method return time=1734888256.555035 sender=:1.51 -> destination=:1.63752
serial=61 reply_serial=2
   array [
      dict entry(
         string "Identifier 0"
         variant             uint16 0
      )
      dict entry(
         string "Number of Identifiers"
         variant             uint16 1
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
```

```
    ]
```

4.4.3.9 MIReadDataStructureControllerInfo

- The Read NVMe-MI Data Structure command requests data that describes information about the NVM Subsystem, the Management Endpoint or the NVMe Controllers
- This command sends Read NVMe-MI Data Structure request with Data Structure Type Controller Information
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTRLID	byte	Controller Identifier (CTRLID): This field contains the Controller identifier whose data structure is returned.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
PCI Bus Number	byte	The Controller's PCI Bus Number.
PCI Device ID	uint16	The PCI Device ID for the specified Controller.
PCI Device Number	byte	The Controller's PCI Device Number.
PCI Function Number	byte	The Controller's PCI Function Number.
PCI Subsystem Device ID	uint16	The PCI Subsystem Device ID for the specified Controller.
PCI Subsystem Vendor ID	uint16	The PCI Subsystem Vendor ID for the specified Controller.
PCI Vendor ID	uint16	The PCI Vendor ID for the specified Controller.
PCIe Routing ID Information	byte	This field provides additional data about the PCI Express Routing ID (PRI) for the specified Controller.
Port Identifier	byte	This field specifies the PCIe Port Identifier with which the Controller is associated.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIReadDataStructureControllerInfo \
byte:105 byte:0 byte:0
method return time=1734888704.523789 sender=:1.51 -> destination=:1.67770
serial=63 reply_serial=2
   array [
      dict entry(
         string "PCI Bus Number"
         variant             byte 0
      )
      dict entry(
         string "PCI Device ID"
```

```
            variant             uint16 9472
         )
         dict entry(
            string "PCI Device Number"
            variant             byte 0
         )
         dict entry(
            string "PCI Function Number"
            variant             byte 0
         )
         dict entry(
            string "PCI Subsystem Device ID"
            variant             uint16 0
         )
         dict entry(
            string "PCI Subsystem Vendor ID"
            variant             uint16 7062
         )
         dict entry(
            string "PCI Vendor ID"
            variant             uint16 7062
         )
         dict entry(
            string "PCIe Routing ID Information"
            variant             byte 0
         )
         dict entry(
            string "Port Identifier"
            variant             byte 0
         )
         dict entry(
            string "Status"
            variant             byte 0
         )
      ]
```

4.4.3.10 MIReadDataStructureOptionalCmd

- The Read NVMe-MI Data Structure command requests data that describes information about the NVM Subsystem, the Management Endpoint or the NVMe Controllers
- This command sends Read NVMe-MI Data Structure request with Data Structure Type Optional Commands Supported
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTRLID	byte	Controller Identifier (CTRLID): This field contains the Controller identifier whose data structure is returned.

- Response

Field	Type	Description

Status	byte	Refer to NVMe-MI response status.
Number of Commands	uint16	This field contains the number of optionally supported commands in the list. A value of 0h indicates there are no commands in the list.
Command 0 Type	byte	This field contains the Command Type for the first optionally supported command or 0h if the list is empty.
Command 0 Opcode	byte	This field contains the Command Opcode for the first optionally supported command or 0h if the list is empty.
... Type	byte	...
... Opcode	byte	...
Command N Type	byte	This field contains the Command Type for the N+1 optionally supported command, if applicable.
Command N Opcode	byte	This field contains the Command Opcode for the N+1 optionally supported command, if applicable.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIReadDataStructureOptionalCmd \
byte:105 byte:0 byte:0
method return time=1734889738.728331 sender=:1.51 -> destination=:1.77373
serial=65 reply_serial=2
   array [
      dict entry(
         string "Command 0 Opcode"
         variant             byte 9
      )
      dict entry(
         string "Command 0 Type"
         variant             byte 2
      )
      dict entry(
         string "Command 1 Opcode"
         variant             byte 16
      )
      dict entry(
         string "Command 1 Type"
         variant             byte 2
      )
      dict entry(
         string "Command 10 Opcode"
         variant             byte 130
      )
      dict entry(
         string "Command 10 Type"
         variant             byte 2
      )
      dict entry(
         string "Command 2 Opcode"
         variant             byte 17
      )
      dict entry(
         string "Command 2 Type"
         variant             byte 2
      )
      dict entry(
         string "Command 3 Opcode"
         variant             byte 20
```

```
        )
        dict entry(
           string "Command 3 Type"
           variant             byte 2
        )
        dict entry(
           string "Command 4 Opcode"
           variant             byte 128
        )
        dict entry(
           string "Command 4 Type"
           variant             byte 2
        )
        dict entry(
           string "Command 5 Opcode"
           variant             byte 0
        )
        dict entry(
           string "Command 5 Type"
           variant             byte 4
        )
        dict entry(
           string "Command 6 Opcode"
           variant             byte 132
        )
        dict entry(
           string "Command 6 Type"
           variant             byte 2
        )
        dict entry(
           string "Command 7 Opcode"
           variant             byte 13
        )
        dict entry(
           string "Command 7 Type"
           variant             byte 2
        )
        dict entry(
           string "Command 8 Opcode"
           variant             byte 21
        )
        dict entry(
           string "Command 8 Type"
           variant             byte 2
        )
        dict entry(
           string "Command 9 Opcode"
           variant             byte 129
        )
        dict entry(
           string "Command 9 Type"
           variant             byte 2
        )
        dict entry(
           string "Number of Commands"
           variant             uint16 11
        )
        dict entry(
           string "Status"
           variant             byte 0
        )
```

```
]
```

4.4.3.11 MISubsystemHealthStatusPoll

- The NVM Subsystem Health Status Poll command is used to efficiently determine changes in health status attributes associated with the NVM Subsystem
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CS	boolean	Clear Status (CS): When this bit is set to 1, the state of reported Composite Controller Status is cleared.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Composite Controller Status Available Spare	byte	This bit is set to '1' when the Available Spare bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Composite Temperature Change	byte	This bit is set to '1' when the Composite Temperature field in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Controller Enable Change Occurred	byte	This bit is set to '1' when the Controller Enable Change Occurred bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Controller Fatal Status	byte	This bit is set to '1' when the Controller Fatal Status bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Controller Status Change	byte	This bit is set to '1' when the Controller Status field in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Critical Warning	byte	This bit is set to '1' when the Critical Warning bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Firmware Activated	byte	This bit is set to '1' when the Firmware Activated bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status NVM Subsystem Reset Occurred	byte	This bit is set to '1' when the value of the NVM Subsystem Reset Occurred (CSTS.NSSRO) bit transitions from a '0' to a '1' in one or more Controllers in the NVM Subsystem.

Composite Controller Status Namespace Attribute Changed	byte	This bit is set to '1' when the Namespace Attribute Changed bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Percentage Used	byte	This bit is set to '1' when the Percentage Used field in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Ready	byte	This bit is set to '1' when the Ready bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Controller Status Shutdown Status	byte	This bit is set to '1' when the Shutdown Status bit in the Controller Health Status Changed Flags transitions from '0' to '1' in one or more Controllers in the NVM Subsystem.
Composite Temperature	string	This field indicates the current temperature in degrees Celsius.
NVM Subsystem Status Drive Functional	byte	This bit is set to '1' to indicate an NVM Subsystem is functional. If cleared to '0', then there is an unrecoverable failure detected in the NVM Subsystem.
NVM Subsystem Status Port 0 PCIe Link Active	byte	This bit is set to '1' to indicate the first port's PCIe link is up (i.e., the Data Link Control and Management State Machine is in the DL_Active state). If cleared to '0', then the PCIe link is down.
NVM Subsystem Status Port 1 PCIe Link Active	byte	This bit is set to '1' to indicate the second port's PCIe link is up. If cleared to '0', then the second port's PCIe link is down or not present.
NVM Subsystem Status Reset Not Required	byte	This bit is set to '1' to indicate the NVM Subsystem does not need a reset to resume normal operation. If cleared to '0' then the NVM Subsystem has experienced an error that prevents continued normal operation. A Controller Level Reset is required to resume normal operation.
Percentage Drive Life Used	byte	Contains a vendor specific estimate of the percentage of NVM Subsystem NVM life used based on the actual usage and the manufacturer's prediction of NVM life.
Smart Warnings Read Only	byte	If set to '0', then the media has been placed in read only mode.
Smart Warnings Reliability Degraded	byte	If set to '0', then the NVM subsystem reliability has been degraded due to significant media related errors or any internal error that degrades NVM subsystem reliability.
Smart Warnings Spare Threshold	byte	If set to '0', then the available spare capacity has fallen below the threshold.
Smart Warnings Temperature Above or Under Threshold	byte	If set to '0', then a temperature is: a) greater than or equal to an over temperature threshold; or b) less than or equal to an under temperature threshold,
Smart Warnings Volatile Memory Backup Failed	byte	If set to '0', then the volatile memory backup device has failed. This field is only valid if the controller has a volatile memory backup solution.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
```

```
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MISubsystemHealthStatusPoll \
byte:105 byte:0 boolean:false
method return time=1734918183.305618 sender=:1.51 -> destination=:1.337559
serial=67 reply_serial=2
   array [
      dict entry(
         string "Composite Controller Status Available Spare"
         variant             byte 1
      )
      dict entry(
         string "Composite Controller Status Composite Temperature Change"
         variant             byte 1
      )
      dict entry(
         string "Composite Controller Status Controller Enable Change
                Occurred"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Controller Fatal Status"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Controller Status Change"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Critical Warning"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Firmware Activated"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status NVM Subsystem Reset Occurred"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Namespace Attribute Changed"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Percentage Used"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Ready"
         variant             byte 0
      )
      dict entry(
         string "Composite Controller Status Shutdown Status"
         variant             byte 0
      )
      dict entry(
         string "Composite Temperature"
         variant             string "30C"
      )
      dict entry(
         string "NVM Subsystem Status Drive Functional"
         variant             byte 1
      )
```

```
      dict entry(
         string "NVM Subsystem Status Port 0 PCIe Link Active"
         variant             byte 1
      )
      dict entry(
         string "NVM Subsystem Status Port 1 PCIe Link Active"
         variant             byte 0
      )
      dict entry(
         string "NVM Subsystem Status Reset Not Required"
         variant             byte 1
      )
      dict entry(
         string "Percentage Drive Life Used"
         variant             byte 0
      )
      dict entry(
         string "Smart Warnings Read Only"
         variant             byte 1
      )
      dict entry(
         string "Smart Warnings Reliability Degraded"
         variant             byte 1
      )
      dict entry(
         string "Smart Warnings Spare Threshold"
         variant             byte 1
      )
      dict entry(
         string "Smart Warnings Temperature Above or Under Threshold"
         variant             byte 1
      )
      dict entry(
         string "Smart Warnings Volatile Memory Backup Failed"
         variant             byte 1
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.12 MIControllerHealthStatusPoll

- The Controller Health Status Poll command is used to efficiently determine changes in health status attributes associated with one or more Controllers in the NVM Subsystem
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
ALL	boolean	Report All (ALL): When this bit is set to '1', a Controller Health Data Structure is returned for Controllers regardless of the status of the Controller Health Status Changed Flags (i.e., it is as though all the bits are set in Controller Health Status Changed Flags).

INCVF	boolean	Include SR-IOV Virtual Functions (INCVF): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers associated with SR-IOV Virtual Functions (VFs).
INCPF	boolean	nclude SR-IOV Physical Functions (INCPF): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers associated with SR-IOV Physical Functions (PFs).
INCF	boolean	Include PCI Functions (INCF): When this bit is set to 1, a Controller Health Status Data Structure is returned for Controllers associated with a non SR-IOV PCI Function.
MAXRENT	byte	Maximum Response Entries (MAXRENT): This field specifies the maximum number of Controller Health Data Structure entries that may be returned in the completion. This is a 0's based field. The maximum number of entries is 255. Specifying 256 entries is interpreted as an Invalid Parameter.
SCTLID	uint16	Starting Controller ID (SCTLID): This field specifies the Controller ID of the first Controller whose Controller Health Data Structure may be returned.
CCF	boolean	Clear Changed Flags (CCF): When this bit is set to 1, the Controller Health Status Changed Flags are cleared in Controllers whose Controller Health Data Structure is contained in the Response Data.
CWARN	boolean	Critical Warning (CWARN): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers with the Critical Warning bit set in their Controller Health Status Changed Flags.
SPARE	boolean	Available Spare (SPARE): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers with the Available Spare bit set in their Controller Health Status Changed Flags.
PDLU	boolean	Percentage Used (PDLU): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers with the Percent Used bit set in their Controller Health Status Changed Flags.
CTEMP	boolean	Composite Temperature Changes (CTEMP): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers with the Composite Temperature bit set in their Controller Health Status Changed Flags.
CSTS	boolean	Controller Status Changes (CSTS): When this bit is set to 1, a Controller Health Data Structure is returned for Controllers with the Ready, Controller Fatal Status, Shutdown Status, NVM Subsystem Reset Occurred, Controller Enable Change Occurred, Namespace Attribute Changed, or Firmware Activated bit set in their Controller Health Status Changed Flags.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Controller 0 Available Spare	uint16	This field contains a normalized percentage (0 to 100%) of the remaining spare capacity available.
Controller 0 Composite Temperature	byte	This field contains a value corresponding to a temperature in degrees Kelvin that represents the current composite temperature of the Controller and namespace(s) associated with that Controller.
Controller 0 Critical Warning Read Only	byte	This bit is set to '1' when the media has been placed in read only mode.
Controller 0 Critical Warning Reliability Degraded	byte	This bit is set to '1' when NVM Subsystem reliability has been degraded due to significant media related errors or an internal error.

Controller 0 Critical Warning Spare Threshold	byte	This bit is set to '1' when the available spare has fallen below the available spare threshold.
Controller 0 Critical Warning Temperature Above or Under Threshold	byte	This bit is set to '1' when a temperature is above an over temperature threshold or below an under temperature threshold.
Controller 0 Critical Warning Volatile Memory Backup Failed	byte	This bit is set to '1' when the volatile memory backup device has failed.
Controller 0 Identifier	uint16	This field specifies the Controller Identifier with which the data contained in this data structure is associated.
Controller 0 Percentage Used	byte	This field contains a vendor specific estimate of the percentage of NVM Subsystem life used based on the actual usage and the manufacturer's prediction of NVM life.
Controller 0 Status Controller Enable Change Occurred	byte	This bit is set to '1' when the Enable bit (refer to CC.EN in the NVM Express specification) changes state.
Controller 0 Status Controller Fatal Status	byte	This bit corresponds to the value of the Controller Fatal Status.
Controller 0 Status Firmware Activated	byte	This bit is set to '1' when a new firmware image is activated. Firmware activation is described in the NVM Express specification. The reset value of this field is '1' if a reset caused a new firmware image to be activated.
Controller 0 Status NVM Subsystem Reset Occurred	byte	This bit corresponds to the value of the NVM Subsystem Reset Occurred (refer to CSTS.NSSRO in the NVM Express specification) bit.
Controller 0 Status Namespace Attribute Changed	byte	This bit is set to '1' when a change occurs in the Identify Namespace data structure for one or more namespaces.
Controller 0 Status Ready	byte	This bit corresponds to the value of the Ready (refer to CSTS.RDY in the NVM Express specification) bit.
Controller 0 Status Shutdown Status	byte	This field corresponds to the value of the Shutdown Status (refer to CSTS.SHST in the NVM Express specification) field.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIControllerHealthStatusPoll \
byte:105 byte:0 boolean:true boolean:false boolean:false boolean:true \
byte:254 uint16:0 boolean:false boolean:true boolean:true boolean:true \
boolean:true boolean:true
method return time=1734921700.820192 sender=:1.51 -> destination=:1.369646
serial=71 reply_serial=2
   array [
      dict entry(
         string "Controller 0 Available Spare"
         variant             byte 100
      )
      dict entry(
         string "Controller 0 Composite Temperature"
         variant             uint16 304
      )
```

```
        dict entry(
           string "Controller 0 Critical Warning Read Only"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Critical Warning Reliability Degraded"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Critical Warning Spare Threshold"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Critical Warning Temperature Above or Under
                  Threshold"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Critical Warning Volatile Memory Backup
                  Failed"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Identifier"
           variant             uint16 0
        )
        dict entry(
           string "Controller 0 Percentage Used"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Controller Enable Change Occurred"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Controller Fatal Status"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Firmware Activated"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status NVM Subsystem Reset Occurred"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Namespace Attribute Changed"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Ready"
           variant             byte 0
        )
        dict entry(
           string "Controller 0 Status Shutdown Status"
           variant             byte 0
        )
        dict entry(
           string "Status"
           variant             byte 0
        )
```

```
   ]
```

4.4.3.13 MIConfigurationSetI2CFreq

- The Configuration Set command allows the Management Controller to modify the current configuration of a Management Endpoint
- The SMBus/I2C Frequency configuration specifies a new frequency for the SMBus port
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
PORTID	byte	This field specifies the port whose SMBus/I2C Frequency is indicated.
FREQ	byte	This field specifies the new frequency for the specified SMBus/I2C port. 0h: Reserved 1h: 100 kHz 2h: 400 kHz 3h: 1 MHz 4h – Fh: Reserved

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIConfigurationSetI2CFreq \
byte:105 byte:0 byte:1 byte:1
method return time=1734923595.695064 sender=:1.51 -> destination=:1.387072
serial=79 reply_serial=2
   array [
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.14 MIConfigurationSetHealthStatusChange

- The Configuration Set command allows the Management Controller to modify the current configuration of a Management Endpoint
- This Configuration Identifier is used to clear selected status bits in the Composite Controller Status field of the NVM Subsystem Health Data Structure returned by the NVM Subsystem Health Status Poll command
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CWARN	boolean	Critical Warning (CWARN): When this bit is set to '1', bit 12 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
SPARE	boolean	Available Spare (SPARE): When this bit is set to '1', bit 11 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
PDLU	boolean	Percentage Used (PDLU): When this bit is set to '1', bit 10 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
CTEMP	boolean	Composite Temperature (CTEMP): When this bit is set to '1', bit 9 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
CSCHNG	boolean	Controller Status Change (CSCHNG): When this bit is set to '1', bit 8 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
FA	boolean	Firmware Activated (FA): When this bit is set to '1', bit 7 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
NAC	boolean	Namespace Attribute Changed (NAC): When this bit is set to '1', bit 6 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
CECO	boolean	Controller Enable Change Occurred (CECO): When this bit is set to '1', bit 5 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
NSSRO	boolean	NVM Subsystem Reset Occurred (NSSRO): When this bit is set to '1', bit 4 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
SHST	boolean	Shutdown Status (SHST): When this bit is set to '1', bit 2 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
CFS	boolean	Controller Fatal Status (CFS): When this bit is set to '1', bit 1 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.
RDY	boolean	Ready (RDY): When this bit is set to '1', bit 0 in the Composite Controller Status field of the NVM Subsystem Health Data Structure is cleared to '0'.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIConfigurationSetHealthStatusChange\
byte:105 byte:0 boolean:true boolean:false boolean:false boolean:false \
boolean:false boolean:false boolean:true boolean:false boolean:false \
boolean:false boolean:false boolean:false
```

```
method return time=1734924295.291843 sender=:1.51 -> destination=:1.393357
serial=83 reply_serial=2
   array [
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.15 MIConfigurationSetMCTPTUSize

- The Configuration Set command allows the Management Controller to modify the current configuration of a Management Endpoint
- The MCTP Transmission Unit Size configuration specifies a new MCTP Transmission Unit Size for the specified Port Identifier
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
PORTID	byte	This field specifies the port whose MCTP Transmission Unit Size is specified.
MTUS	uint16	MCTP Transmission Unit Size: This field contains the MCTP Transmission Unit Size in bytes to be used by the port.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIConfigurationSetMCTPTUSize \
byte:105 byte:0 byte:1 uint16:64
method return time=1734924849.958574 sender=:1.51 -> destination=:1.398578
serial=89 reply_serial=2
   array [
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.16 MIConfigurationGetI2CFreq

- The Configuration Get command allows the Management Controller to read the current configuration of a Management Endpoint

- The SMBus/I2C Frequency configuration indicates the current frequency of the SMBus port, if applicable
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
PORTID	byte	Port Identifier: This field specifies the port whose SMBus/I2C Frequency is indicated.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
SMBus/I2C Frequency	string	SMBus/I2C Frequency: The current frequency of the SMBus/I2C. The default value for this field following a reset or power cycle is 1h, if SMBus is supported. 0h: SMBus is not supported or is disabled 1h: 100 kHz 2h: 400 kHz 3h: 1 MHz 4h – Fh: Reserved

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIConfigurationGetI2CFreq \
byte:105 byte:0 byte:0
method return time=1734925123.872419 sender=:1.51 -> destination=:1.400986
serial=93 reply_serial=2
   array [
      dict entry(
         string "SMBus/I2C Frequency"
         variant             string "100 kHz"
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.17 MIConfigurationGetMCTPTUSize

- The Configuration Get command allows the Management Controller to read the current configuration of a Management Endpoint
- The MCTP Transmission Unit Size configuration indicates the current MCTP Transmission Unit Size of the specified Port Identifier
- Request parameter

Field	Type	Description

EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
PORTID	byte	Port Identifier: This field specifies the port whose MCTP Transmission Unit Size is indicated.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
MCTP Transmission Unit Size	uint16	This field contains the MCTP Transmission Unit Size in bytes to be used by the port. The default value for this field following a reset or power cycle is 40h (64).

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIConfigurationGetMCTPTUSize \
byte:105 byte:0 byte:1
method return time=1734925509.775074 sender=:1.51 -> destination=:1.404616
serial=95 reply_serial=2
   array [
      dict entry(
         string "MCTP Transmission Unit Size"
         variant             uint16 64
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.18 MIVPDRead

- The VPD Read command is used to read the Vital Product Data. Upon successful completion of the VPD Read command, the specified portion of the VPD contents is returned in the Response Data
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
DOFST	uint16	Data Offset (DOFST): This field specifies the starting offset, in bytes, into the VPD data that is contained in the Response Message.
DLEN	uint16	Data Length (DLEN): This field specifies the length, in bytes, to be read from the VPD starting at the byte offset specified by DOFST.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
MCTP Transmission Unit Size	uint16	This field contains the MCTP Transmission Unit Size in bytes to be used by the port. The default value for this field following a reset or power cycle is 40h (64).

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIVPDRead \
byte:105 byte:0 uint16:0 uint16:8
method return time=1734931578.439121 sender=:1.51 -> destination=:1.460149
serial=97 reply_serial=2
   array [
      dict entry(
         string "Data"
         variant             array of bytes [
               01 00 00 0f 01 15 00 da
            ]
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.19 MIVPDWrite

- The VPD Write command is used to update the Vital Product Data. After the VPD Write command completes successfully, reading the contents of the FRU Information Device directly or executing a VPD Read command shall return the new VPD contents
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
DOFST	uint16	Data Offset (DOFST): This field specifies the starting offset, in bytes, into the VPD data that is written.
DLEN	uint16	Data Length (DLEN): This field specifies the length, in bytes, to be written to the VPD starting at the byte offset specified by DOFST.
Data	byte array	The data to be written into the VPD.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.MIVPDWrite \
byte:105 byte:0 uint16:1 uint16:2 array:byte:0x00,0x00
method return time=1734933770.579309 sender=:1.51 -> destination=:1.480122
serial=104 reply_serial=2
   array [
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.20 ACIdentify

- The Identify command returns a data buffer that describes information about the NVM subsystem, the controller or the namespace(s)
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTLID	uint16	Controller Identifier (CNTID): This field specifies the controller identifier used as part of some Identify operations.
NSID	uint32	Namespace Identifier (NSID): This field specifies the namespace that this command applies to.
DOFST	uint32	Data Offset (DOFST): For commands that transmit data from the Management Controller to the Management Endpoint (i.e., the NVMe Data field in the Request Message has non-zero length) or do not transmit data, this field shall be cleared to '0'. If this field is not 0h, then the Management Endpoint shall return an error response with status Invalid Parameter. For commands that transmit data from the Management Endpoint to the Management Controller (i.e., the NVMe Data field in the Response Message has non-zero length), this field specifies the starting offset, in bytes, into the completion data contained in the Response Message. Bits 0 and 1 of this field shall be cleared to '0'.
DLEN	uint32	Data Length (DLEN): For commands that do not transmit data in neither the Request Message nor Response Message, this field shall be cleared to 0h. If this field is not 0h, then the Management Endpoint shall return an error response with status Invalid Parameter. For commands that transmit data from the Management Controller to the Management Endpoint (i.e., the NVMe Data field in the Request Message has non-zero length), this field specifies the length, in bytes, of the data contained in the Request Message. For commands that transmit data from the Management Endpoint to the Management Controller (i.e., the NVMe Data field in the Response Message has non-zero length), this field specifies the length, in bytes, of the data contained in the

		Response Message. Bits 0 and 1 of this field shall be cleared to '0'. This field shall be less than or equal to 4096.
CNS	byte	Controller or Namespace Structure (CNS): This field specifies the information to be returned to the host.
CNTID	uint16	Controller Identifier (CNTID): This field specifies the controller identifier used as part of some Identify operations.
NVMSETID	uint16	NVM Set Identifier (NVMSETID): This field specifies the identifier of the NVM Set. This field is used for Identify operations with a CNS value of 04h. This field should be cleared to 0h for Identify operations with other CNS values.
UUID Index	byte	UUID Index: If this field is set to a non-zero value, then the value of this field is the index of a UUID in the UUID List that is used by the command. If this field is cleared to 0h, then no UUID index is specified.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Data	byte array	Response raw data

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ACIdentify \
byte:105 byte:0 uint16:0 uint32:0 uint32:0 uint32:72 byte:1 uint16:0 \
uint16:0 byte:0
method return time=1734935288.696039 sender=:1.51 -> destination=:1.494056
serial=108 reply_serial=2
   array [
      dict entry(
         string "Data"
         variant             array of bytes [
              96 1b 96 1b 41 30 36 41 32 36 30 31 20 20 20 20 20 20 20 20 20
              20 20 20 57 55 53 34 43 36 34 33 32 44 53 50 33 58 31 20 20 20
              20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20
              20 52 32 32 31 30 30 31 30
            ]
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.21 ACGetFeatures

- The Get Features command retrieves the attributes of the Feature specified
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.

CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTLID	uint16	Controller Identifier (CNTID): This field specifies the controller identifier used as part of some Identify operations.
NSID	uint32	Namespace Identifier (NSID): This field specifies the namespace that this command applies to.
SEL	byte	Select (SEL): This field specifies which value of the attributes to return in the provided data: 000b: Current 001b: Default 010b: Saved 011b: Supported capabilities 100b to 111b: Reserved
FID	byte	Feature Identifier (FID): This field specifies the identifier of the Feature for which to provide data.
UUID Index	byte	UUID Index: If this field is set to a non-zero value, then the value of this field is the index of a UUID in the UUID List that is used by the command. If this field is cleared to 0h, then no UUID index is specified.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Arbitration Burst	byte	Arbitration Burst (AB): Indicates the maximum number of commands that the controller may launch at one time from a particular Submission Queue. The value is expressed as a power of two (e.g., 000b indicates one, 011b indicates eight). A value of 111b indicates no limit.
High Priority Weight	byte	High Priority Weight (HPW): This field defines the number of commands that may be executed from the high priority service class in each arbitration round. This is a 0's based value.
Low Priority Weight	byte	Low Priority Weight (LPW): This field defines the number of commands that may be executed from the low priority service class in each arbitration round. This is a 0's based value.
Medium Priority Weight	byte	Medium Priority Weight (MPW): This field defines the number of commands that may be executed from the medium priority service class in each arbitration round. This is a 0's based value.

- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ACGetFeatures \
byte:105 byte:0 uint16:0 uint32:0 byte:0 byte:1 byte:0
method return time=1734938209.066902 sender=:1.51 -> destination=:1.520847
serial=110 reply_serial=2
   array [
      dict entry(
         string "Arbitration Burst"
         variant             byte 0
      )
```

```
      dict entry(
         string "High Priority Weight"
         variant             byte 0
      )
      dict entry(
         string "Low Priority Weight"
         variant             byte 0
      )
      dict entry(
         string "Medium Priority Weight"
         variant             byte 0
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

4.4.3.22 ACGetLogPage

- The Get Log Page command returns a data buffer containing the log page requested
- Request parameter

Field	Type	Description
EID	byte	EID of target NVMe-MI device.
CSI	byte	Command Slot Identifier (CSI): This field indicates the Command Slot with which the message is associated. 0h: Command Slot 0 1h: Command Slot 1
CTLID	uint16	Controller Identifier (CNTID): This field specifies the controller identifier used as part of some Identify operations.
NSID	uint32	Namespace Identifier (NSID): This field specifies the namespace that this command applies to.
DOFST	uint64	Data Offset (DOFST): For commands that transmit data from the Management Controller to the Management Endpoint (i.e., the NVMe Data field in the Request Message has non-zero length) or do not transmit data, this field shall be cleared to '0'. If this field is not 0h, then the Management Endpoint shall return an error response with status Invalid Parameter. For commands that transmit data from the Management Endpoint to the Management Controller (i.e., the NVMe Data field in the Response Message has non-zero length), this field specifies the starting offset, in bytes, into the completion data contained in the Response Message. Bits 0 and 1 of this field shall be cleared to '0'.
DLEN	uint32	Data Length (DLEN): For commands that do not transmit data in neither the Request Message nor Response Message, this field shall be cleared to 0h. If this field is not 0h, then the Management Endpoint shall return an error response with status Invalid Parameter. For commands that transmit data from the Management Controller to the Management Endpoint (i.e., the NVMe Data field in the Request Message has non-zero length), this field specifies the length, in bytes, of the data contained in the Request Message. For commands that transmit data from the Management Endpoint to the Management Controller (i.e., the NVMe Data field in the Response Message has non-zero length), this field specifies the length, in bytes, of the data contained in the

		Response Message. Bits 0 and 1 of this field shall be cleared to '0'. This field shall be less than or equal to 4096.
RAE	boolean	Retain Asynchronous Event (RAE): This bit specifies when to retain or clear an Asynchronous Event. If this bit is cleared to '0', the corresponding Asynchronous Event is cleared after the command completes successfully. If this bit is set to '1', the corresponding Asynchronous Event is retained (i.e., not cleared) after the command completes successfully. Host software should clear this bit to '0' for log pages that are not used with Asynchronous Events.
UUID Index	byte	UUID Index: If this field is set to a non-zero value, then the value of this field is the index of a UUID in the UUID List that is used by the command. If this field is cleared to 0h, then no UUID index is specified.
LID	byte	Log Page Identifier (LID): This field specifies the identifier of the log page to retrieve.

- Response

Field	Type	Description
Status	byte	Refer to NVMe-MI response status.
Data	byte array	Response raw data.

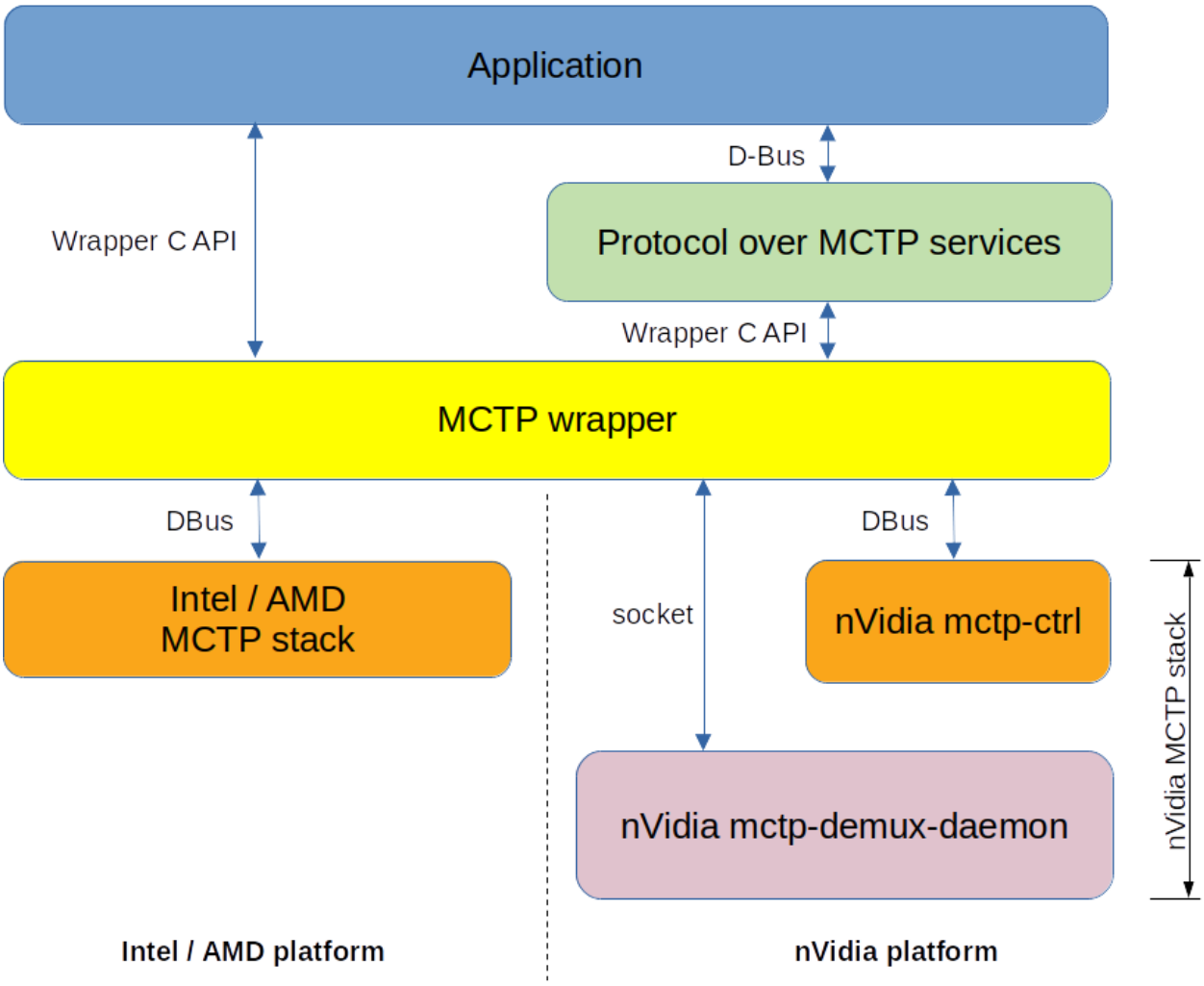
- Example request and response

```
# dbus-send --system --print-reply --dest=xyz.openbmc_project.NVMeMIDaemon \
/xyz/openbmc_project/nvmemi \
xyz.openbmc_project.NVMeMIDaemon.NVMeMI.ACGetLogPage \
byte:105 byte:0 uint16:0 uint32:0xFFFFFFFF uint64:0 uint32:64 boolean:true \
byte:0 byte:2
method return time=1734939475.638849 sender=:1.51 -> destination=:1.532432
serial=116 reply_serial=2
   array [
      dict entry(
         string "Data"
         variant             array of bytes [
              00 2f 01 64 0a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
              00 00 00 00 00 00 00 00 00 00 00 c4 cd 00 00 00 00 00 00 00
              00 00 00 00 00 00 36 0a 01 00 00 00 00 00 00 00 00 00 00 00
              00
            ]
      )
      dict entry(
         string "Status"
         variant             byte 0
      )
   ]
```

5. MCTP Wrapper

5.1 Introduction

To decrease MCTP application development complexity on difference MCTP stack, Insyde MCTP wrapper provides unified C functions to utilize MCTP functions, including getting endpoint information, sending MCTP messages, and receiving MCTP messages.



Like the figure above, with MCTP wrapper support, there is no need to maintain different versions of applications just for applying different interfaces of MCTP stacks.

5.2 MCTP Wrapper API

5.2.1 MCTP_BINDING_ID

- These enum numbers are used to initialize the wrapper to access via which transport binding

Name	Value	Description

MCTP_OVER_SMBUS	1	Use MCTP over SMBus
MCTP_OVER_PCIE	2	Use MCTP over PCIe

5.2.2 MCTP_MESSAGE_TYPE

- These enum numbers are used assign which message type to transmission by the wrapper

Name	Value	Description
MCTP_MSG_TYPE_MCTP_CONTROL	00h	MCTP Control
MCTP_MSG_TYPE_PLDM	01h	PLDM
MCTP_MSG_TYPE_NCSI	02h	NC-SI
MCTP_MSG_TYPE_ETHERNET_OVER_MCTP	03h	Ethernet
MCTP_MSG_TYPE_NVMEMI	04h	NVMe-MI
MCTP_MSG_TYPE_SPDM	05h	SPDM
MCTP_MSG_TYPE_VENDOR_DEFINED_PCIE	7Eh	Vendor Defined – PCI
MCTP_MSG_TYPE_VENDOR_DEFINED_IANA	7Fh	Vendor Defined – IANA

5.2.3 MCTPW_EVENT

- These enum numbers are to indicate the event type of device updated callback

Name	Value	Description
MCTPW_EVENT_EP_ADDED	00h	A new endpoint is added
MCTPW_EVENT_EP_REMOVED	01h	An endpoint is removed

5.2.4 MCTPW_STATUS

- These defined number are used to indicate error state of wrapper API

Name	Value	Description
MCTPW_STATUS_OK	0	No error
MCTPW_STATUS_INVALID_PARAM	-1	Invalid input parameter
MCTPW_STATUS_INTERNAL_ERROR	-2	Error returned from wrapper itself
MCTPW_STATUS_EXCEPTION	-3	Error due to exception
MCTPW_STATUS_SERVICE_ACCESS_PROTOCOL_ERROR	-4	Error when access the interface of MCTP service
MCTPW_STATUS_SERVICE_TRANSFER_ERROR	-5	Error when transferring data to MCTP service
MCTPW_STATUS_SERVICE_RETURN_ERROR	-6	Error when receive an error from MCTP service

5.2.5 MCTPW_LOG

- These defined numbers are used to set the level of the debug log to be printed

Name	Value	Description

MCTPW_LOG_ERR	3	Only print log its level is not lower than this value
MCTPW_LOG_WARNING	4	Only print log its level is not lower than this value
MCTPW_LOG_NOTICE	5	Only print log its level is not lower than this value
MCTPW_LOG_INFO	6	Only print log its level is not lower than this value
MCTPW_LOG_DEBUG	7	Only print log its level is not lower than this value

5.2.6 Function

5.2.6.1 mctpw_new

- Initialize a wrapper handle to operate functions of wrapper, and specify the transport binding and message type to use

```
int mctpw_new(MCTP_BINDING_ID binding,
              MCTP_MESSAGE_TYPE msg_type,
              void **handle);
```

Parameter	in/out	Description
binding	in	Select the binding type of MCTP service. Refer to MCTP_BINDING_ID.
msg_type	in	The type of messages will be handled. Refer to MCTP_MESSAGE_TYPE.
handle	out	MCTP wrapper context data.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code

5.2.6.2 mctpw_set_device_update_callback

- Setup a callback that will be called when any endpoint is added or removed

```
int mctpw_set_device_update_callback(void *handle,
                                     mctpw_device_update_cb device_update_cb);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
device_update_cb	in	The callback function to be called when endpoints are added or removed. Refer to mctpw_device_update_cb.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code

5.2.6.3 mctpw_set_rx_message_callback

- Setup a callback to receive MCTP message asynchronously

```
int mctpw_set_rx_message_callback(void *handle,
                                  mctpw_rx_message_cb rx_cb);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
rx_cb	in	The callback function to be called when receiving MCTP message. Refer to mctpw_rx_message_cb.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code

5.2.6.4 mctpw_set_user_data

- Setup a pointer that points to arbitrary data that will be passed into the mctpw_rx_message_cb

```
void mctpw_set_user_data(void *handle,
                         void *userdata);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
userdata	in	Pointer of the userdata to be passed into mctpw_rx_message_cb.

5.2.6.5 mctpw_set_log_level

- Setup level of debug log to be printed

```
void mctpw_set_log_level(void *handle,
                         int level);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
level	in	Debug log level. Refer to MCTPW_LOG.

5.2.6.6 mctpw_get_ep_list

- Get an array of bytes that each byte presents an endpoint. Only the endpoints supporting the specified message type will be in the array

```
int mctpw_get_ep_list(void *handle,
                      MCTP_MESSAGE_TYPE type,
                      uint8_t *eids,
                      size_t *eid_list_size);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
type	in	Only return the endpoints which support this message type.
eids	in	The buffer Endpoint IDs will be stored in.

eid_list_size	in/out	in: Tell MCTP wrapper the buffer length of eids. out: Returned data length will be stored in eid_list_size when this API returns.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code

5.2.6.7 mctpw_send

- Send an MCTP message payload

```
int mctpw_send(void *handle,
               uint8_t dst_eid,
               uint8_t msg_tag,
               bool tag_owner,
               uint8_t *payload,
               size_t payload_len);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
dst_eid	in	Destination endpoint ID.
msg_tag	in	Message tag of the MCTP message.
tag_owner	in	Tag owner of MCTP message.
payload	in	Array of MCTP message payload bytes.
payload_len	in	Length of MCTP message payload.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code
- Note
    - Caller can only send messages with type specified in mctpw_new

5.2.6.8 mctpw_send_receive

- Send an MCTP message payload and wait until receiving response or timeout

```
int mctpw_send_receive(void *handle,
                       uint8_t dst_eid,
                       uint8_t *payload,
                       size_t payload_len,
                       uint16_t timeout_ms,
                       uint8_t *rx_buffer,
                       size_t *rx_len);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.
dst_eid	in	Destination endpoint ID.
payload	in	Array of MCTP message payload bytes.

payload_len	in	Length of MCTP message payload.
timeout_ms	in	Timeout of waiting in milliseconds.
rx_buffer	out	Pointer to the buffer to store response data.
rx_len	in/out	in: Length of rx_buffer. out: Response data length will be stored in rx_len when API returns.

- On success, it returns MCTPW_STATUS_OK, otherwise other MCTPW_STATUS code
- Note
    - Caller can only send messages with type specified in mctpw_new

5.2.6.9 mctpw_delete

- Free the resource of wrapper

```
void mctpw_delete(void *handle);
```

Parameter	in/out	Description
handle	in	MCTP wrapper context data.

5.2.6.10 mctpw_device_update_cb

- This is the callback function that will be called when any endpoint is added or removed
- Set by mctpw_set_device_update_callback

```
typedef void (*mctpw_device_update_cb)(MCTPW_EVENT event,
                                       uint8_t eid,
                                       void *userdata);
```

Parameter	in/out	Description
event	out	Indicate the endpoint is added or removed. Refer to MCTPW_EVENT.
eid	out	The EID of the added or removed endpoint.
userdata	out	Pointer to the data set with mctpw_set_user_data.

5.2.6.11 mctpw_rx_message_cb

- This is the callback function that will be called when receiving MCTP messages
- Set by mctpw_set_rx_message_callback

```
typedef void (*mctpw_rx_message_cb)(uint8_t msg_type,
                                    uint8_t src_eid,
                                    uint8_t msg_tag,
```

	`bool tag_owner,`
	`uint8_t* payload,`
	`size_t payload_len,`
	`void *userdata);`	

Parameter	**in/out**	**Description**
msg_type	out	Message type of the MCTP message.
src_eid	out	EID of the source endpoint.
msg_tag	out	Message tag of the MCTP message.
tag_owner	out	Tag owner of the MCTP message.
payload	out	MCTP message payload.
payload_len	out	Length of the MCTP message payload.
userdata	out	Pointer to the data set with mctpw_set_user_data.

5.3 Example Code

- Please refer to the sample code shows how to use MCTP wrapper
  - For OPF G3
    - <SDK>/meta-insyde/meta-common/meta-share/recipes-insyde/libmctp-wrapper/files/src/mctp_wrapper_test.cpp
  - For OPF G4
    - <SDK>/meta-insyde /meta-common/meta-core/recipes-insyde/libmctp-wrapper/files/src/mctp_wrapper_test.cpp